

KT그룹

AX 전략기획 및 멀티에이전트 서비스 개발

Agentic AI Agentic 워크플로우

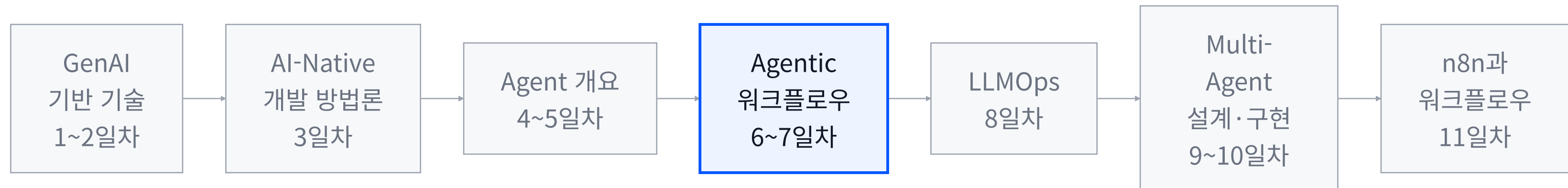


CONTENTS

목차

- 01 상태·노드·조건부 엣지
- 02 체크포인트와 사람 개입
- 03 서브그래프 모듈화
- 04 추상화 스펙트럼과 선택 기준
- 05 MCP 구조와 연결
- 06 커스텀 MCP 서버
- 07 멀티모달 입출력
- 08 이미지 생성 파이프라인

과정 전체 지도



이 강의에서 쓰는 에이전트 최소 지식

그래프 코드와 도해가 되는 여섯 낱말. 낱말마다 뜻과 해당 코드.

도구 호출 루프(tool use loop)

모델이 도구를 고르고, 그 결과를 받아 다시 판단하는 반복 구조. 더 부를 도구가 없으면 종료됨

```
if not res.tool_calls: break
```

bind_tools / tool_calls

모델에 도구 목록을 묶어 두는 위치가 bind_tools, 응답에서 호출 지시를 꺼내는 위치가 tool_calls

```
llm.bind_tools(TOOLS)  
.invoke(msgs).tool_calls
```

MessagesState / add_messages

메시지 이력을 담는 상태 컨테이너와, 새 메시지를 덮어쓰지 않고 뒤에 쌓는 누적 규칙, 즉 리듀서(reducer)

```
messages: Annotated[list, add_messages]
```

구조화 출력(Structured Output)

pydantic 스키마로 응답의 모양을 강제하는 호출 방식. 정해 둔 필드가 그대로 객체로 돌아옴

```
llm.with_structured_output(Email)  
.invoke(text)
```

init_chat_model / litellm

모델 호출이 지나는 진입점. 모델 이름만 바꾸면 다른 모델로 같은 코드가 돌

```
init_chat_model("openai/gpt-5.6-luna",  
model_provider="litellm")
```

.env API 키

키를 코드 밖 파일에 두고 실행 시점에 환경 변수로 읽어 들이는 관용구

```
from dotenv import load_dotenv  
load_dotenv()
```

문제 상황: 배치 처리 중 프로세스 종료

```
=== 배치 처리 (10건) ===
[처리 1/10] LLM 호출 -> 환불은 언제까지...
[처리 2/10] LLM 호출 -> 배송이 3일째...
[처리 3/10] LLM 호출 -> 주차 요금이...
[처리 4/10] LLM 호출 -> 상품이 파손되어...
[처리 5/10] LLM 호출 -> ★무료 이벤트★...
[처리 6/10] LLM 호출 -> 운영시간...
Traceback (most recent call last):
  File "mail_batch.py", line 280,
    in process_one
    raise RuntimeError(
      f"[강제 예외] {n}번째 처리 중 프로세스 사망")
##### exit=1 #####
```

10건 배치, 6건 처리 후 7번째에서 종료

- 종료 코드 `exit=1` . 이 실행에서 뒤따르는 출력은 없음
- 종료 시점까지 쓴 LLM 호출 6회. 결과는 프로세스 메모리와 함께 소멸
- 재실행의 출발점은 1번. 6건분 호출 비용을 다시 지불

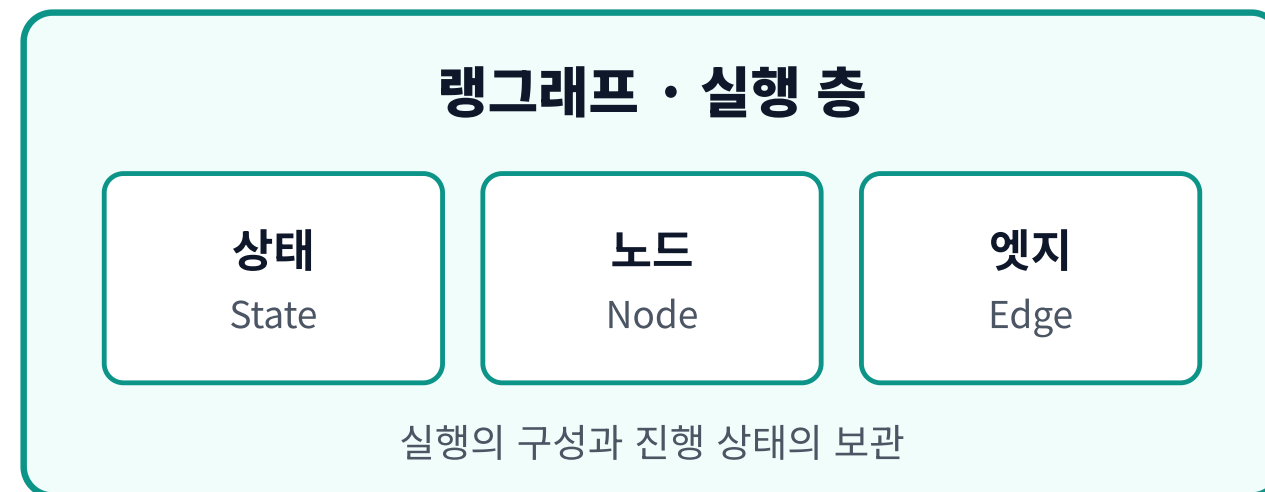
진행한 곳을 저장해 두지 않으면, 재실행의 출발점은 언제나 처음.

문제 상황: 승인 대기가 들어갈 위치

```
def run_hand_loop(question: str, max_turn: int = 5):
    bound = llm.bind_tools(TOOLS)
    messages = [SystemMessage(SYSTEM), HumanMessage(question)]
    trace = []
    for _ in range(max_turn):
        res = bound.invoke(messages)
        messages.append(res)
        if not res.tool_calls:
            break
        for call in res.tool_calls:
            out = TOOL_MAP[call["name"]].invoke(call["args"])
            trace.append((call["name"], call["args"]))
            messages.append(ToolMessage(str(out), tool_call_id=call["id"]))
    return messages[-1].content, trace
```

- 요구사항: 초안이 만들어지면 담당자 승인을 받은 뒤 발송. 승인까지는 몇 시간이 걸림
- 반복문 안에서 그 시간을 기다리면 프로세스가 그동안 점유됨. 대기 중 종료되면 진행분도 함께 소멸
- 반복문 밖으로 빼면 `messages` 와 `trace` 가 함수 지역 변수라 승인 시점에 남아 있지 않음

랭체인 1.0의 실행 층



위층은 아래층 부품을 그대로 쓴다



- 랭그래프(LangGraph)는 랭체인(LangChain) 1.0의 실행 층. 두 이름은 같은 패키지에서 함께 импорт됨
- 모델 호출 · 도구 · 메시지 부품은 그대로 쓰임.
`init_chat_model` · `@tool` · `bind_tools` ·
메시지 객체가 같은 이름으로 등장
- 실행 층이 맡는 것은 부품 교체가 아니라 **실행을 구성하고 진행 상태를 보관하는 일**

```
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END
```

Agentic 워크플로우

상태 · 노드 · 조건부 엣지

전체 구조 / 정의 / 단계별 구성 / 리팩토링

학습 목표

- 상태(State)·노드(Node)·조건부 엣지(Conditional Edge)의 역할을, 그래프 코드를 보며 설명할 수 있습니다.
- 상태 정의 → 노드 준비 → 그래프 구성 → 실행의 절차로, 노드 3개 이하의 그래프를 구성해 실행할 수 있습니다.
- 손으로 짠 도구 호출 루프를 그래프로 옮기면 반복과 종료 판단이 어느 위치로 가는지 코드 두 벌에서 짚고, 그렇게 옮겨서 무엇을 얻는지 설명할 수 있습니다.

▶ 상태·노드·조건부 엣지

체크포인트와 사람 개입
서브그래프 모듈화
추상화 스펙트럼과 선택 기준
MCP 구조와 연결
커스텀 MCP 서버
멀티모달 입출력
이미지 생성 파이프라인

문제상황과 목표

문제

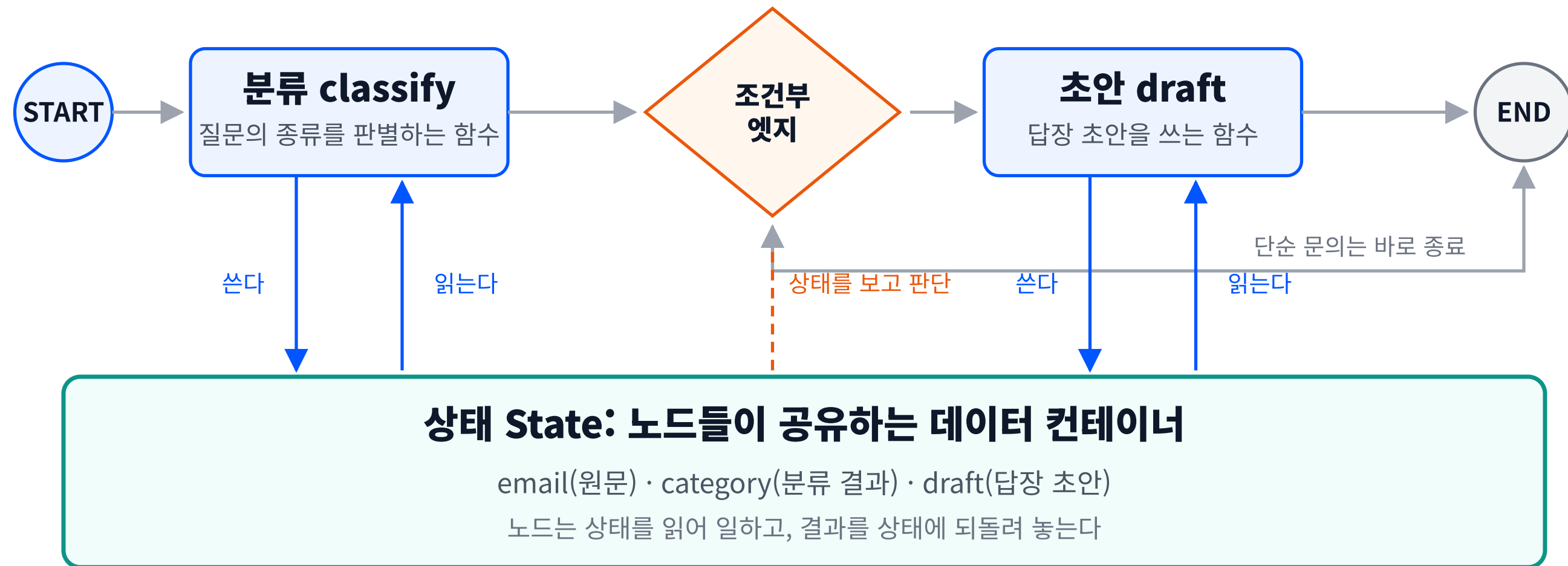
온라인 쇼핑몰 고객센터예요. 환불·배송 문의 메일에 광고 메일이 뒤섞여 들어오는데, 광고는 걸러 내고 나머지는 답장 초안까지 있으면 좋겠어요.

목표

받은 메일을 종류별로 나누고, 광고면 바로 끝내고 아니면 답장 초안을 써 주는 자동 처리 흐름을 만들어봅시다.

랭그래프(LangGraph) 그래프의 전체 골격

흐름: 엣지(Edge)가 순서를 정한다



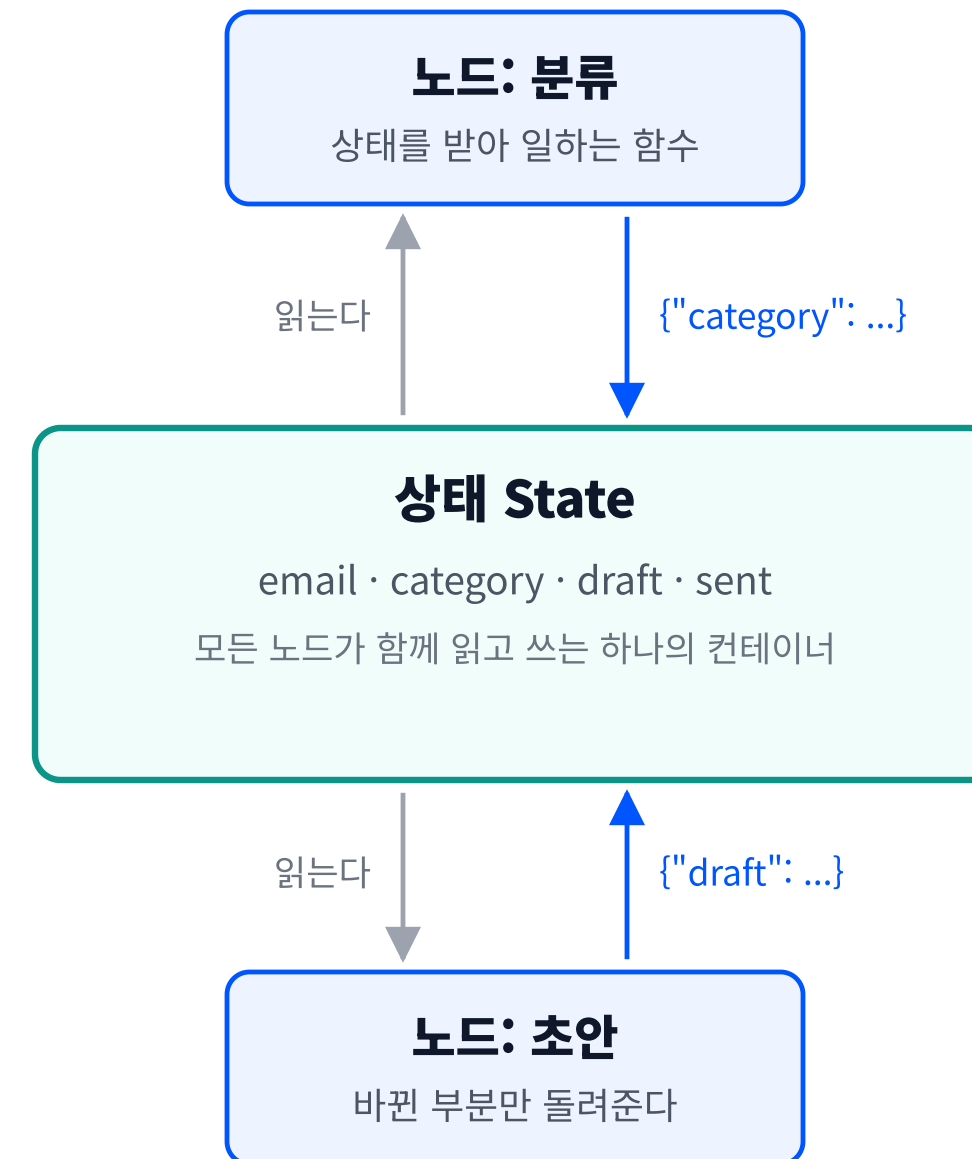
상태를 가운데 두고, 노드가 그 상태를 읽어 일하고 결과를 되돌려 놓으며, 엣지가 순서를 정함.

각 단계별 요구사항

- **분류 classify**: 고정 지침(시스템 프롬프트)과 상태의 메일 본문을 입력으로 → 어느 종류인지 한 단어로 판단해 → 분류 결과를 상태에 쓴다
- **조건부 엣지**: 상태의 분류 결과만 보고 → 스팸이면 END로, 아니면 초안으로 보낸다
- **초안 draft**: 그 종류의 상담원 시스템 프롬프트와 상태의 메일 본문을 입력으로 → 두 문장 답장을 써 → 답장 초안을 상태에 쓴다
- **상태 State**: 메일 본문·분류 결과·답장 초안 세 키를 가지고 → 노드가 읽고 쓰게 두며 → 흐름 내내 하나로 유지된다

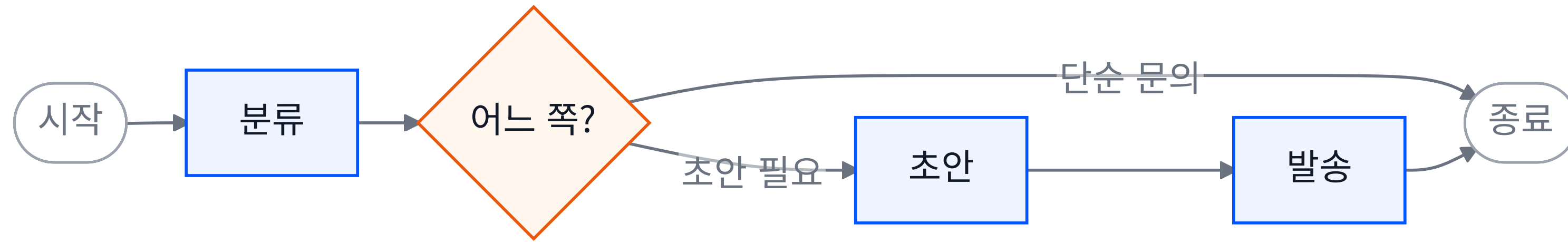
상태(State)와 노드(Node)

- **상태(State)**: 그래프가 도는 동안 모든 노드가 함께 읽고 쓰는 하나의 상태 컨테이너(state container). `TypedDict` 로 어떤 키를 어떤 타입으로 담을지 미리 선언함.
- **노드(Node)**: 상태를 인자로 받아 일하고, 딕셔너리를 돌려주는 함수. 돌려준 딕셔너리가 상태의 해당 키를 덮음.
- 어느 키를 돌려줄지는 노드의 구현 방식에 따라 다름. `{"category": "환불"}` 한 키만 돌려주면 나머지 키는 그대로 남음.



엣지(Edge)와 조건부 엣지(Conditional Edge)

받은 메일을 분류해 답장 초안을 쓰고 발송하는 그래프의 연결.



- **엣지(Edge)**: 노드에서 노드로 가는 고정된 선. `add_edge(a, b)` 호출이 "a가 끝나면 b"를 뜻함
- **조건부 엣지(Conditional Edge)**: 판단 함수가 돌려준 이름으로 다음 노드가 나뉘는 선. 이 분기 구조를 라우팅(routing)이라 부름. `add_conditional_edges(출발 노드, 판단 함수, 매핑)`
- **START·END**: 그래프의 진입점과 종료점을 가리키는 특수 노드. 어떤 함수도 아님

예시 코드가 미리 정의해 둔 변수: PROMPT · SPAM · IN · MODE

```
# PROMPT - classify 노드가 쓰는 분류 지시문
다음 고객 메일을 환불, 배송, 스팸 중
한 단어로만 분류한다.
# SPAM - route 함수가 비교하는 갈림값
"스팸"
# IN - graph.stream에 넣는 입력 메일
{"email": "지난주에 주문한 상품 환불받고 싶습니다."}
# MODE - stream의 출력 방식
"updates"
```

write 함수는 답장을 쓰지 않는 축약본

```
# 전체 코드에서는 draft 함수가 이 지시문으로 답장을 씀
'{분류}' 문의에 답하는 상담원이다.
정중한 답장을 두 문장으로 쓴다.
```

변수	어디에서 쓰나	무엇을 정하는가
PROMPT	classify의 llm.invoke	메일 분류 프롬프트
SPAM	route의 비교식	이 값이면 END로 감
IN	graph.stream 첫 인자	처리할 입력 메일
MODE	stream_mode 인자	바뀐 부분만 보는 방식

1단계: 상태 정의

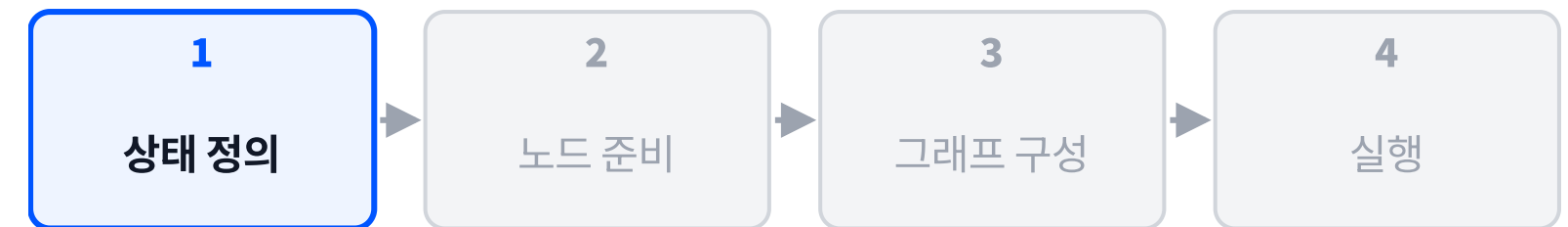
```
class EmailState(TypedDict):
    email: str # 메일 본문
    category: str # 분류 결과
    draft: str # 답장 초안

def classify(s: EmailState) -> dict:
    r = llm.invoke(PROMPT + s["email"])
    return {"category": r.content.strip()}

def write(s): return {"draft": s["category"]}

def route(s: EmailState) -> str:
    if s["category"] == SPAM: return END
    return "reply"

g = StateGraph(EmailState)
g.add_node("classify", classify)
g.add_node("reply", write)
g.add_edge(START, "classify")
g.add_conditional_edges("classify", route)
graph = g.compile()
print(*graph.stream(IN, stream_mode=MODE))
```



EmailState 하나가 이 그래프의 데이터 전부. 노드들이 주고받을 항목은 이 세 키뿐.

TypedDict로 키와 타입을 미리 못 박음. 선언에 없는 키는 상태에 들어가지 않음.

비어 있는 키는 아직 아무 노드도 채우지 않은 키.

2단계: 노드 준비

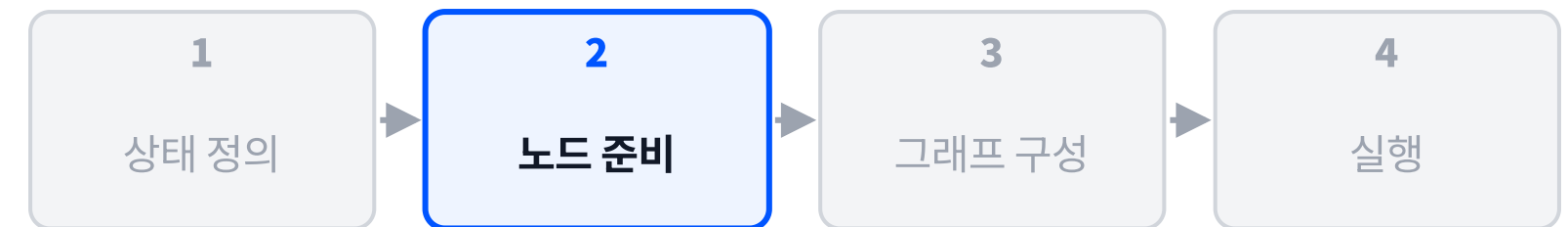
```
class EmailState(TypedDict):
    email: str # 메일 본문
    category: str # 분류 결과
    draft: str # 답장 초안

def classify(s: EmailState) -> dict:
    r = llm.invoke(PROMPT + s["email"])
    return {"category": r.content.strip()}

def write(s): return {"draft": s["category"]}

def route(s: EmailState) -> str:
    if s["category"] == SPAM: return END
    return "reply"

g = StateGraph(EmailState)
g.add_node("classify", classify)
g.add_node("reply", write)
g.add_edge(START, "classify")
g.add_conditional_edges("classify", route)
graph = g.compile()
print(*graph.stream(IN, stream_mode=MODE))
```



classify는 상태를 인자로 받는 평범한 파이썬 함수. 특별한 상속도 데코레이터도 없음.

돌려주는 값은 **{"category": ...}** 한 키뿐. 바뀐 부분만 돌려주면 나머지 키는 그대로 남음.

답장 노드(**write**)도 같은 모양. 상태를 받아 자기 키만 채워 돌려줌.

3단계: 그래프 구성

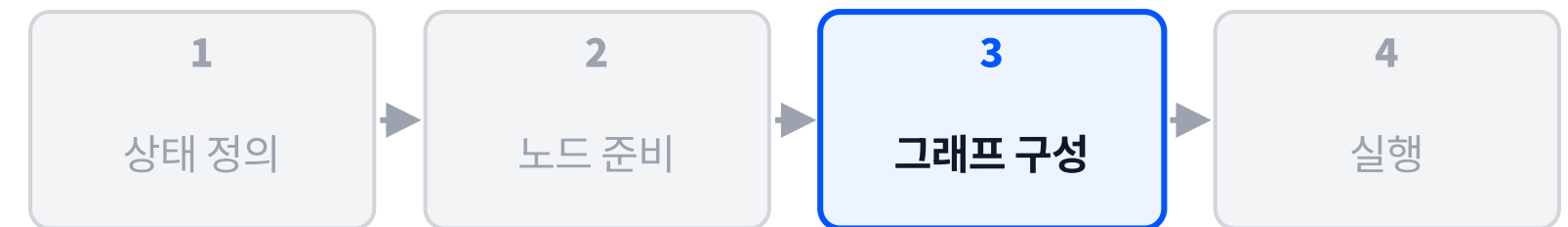
```
class EmailState(TypedDict):
    email: str # 메일 본문
    category: str # 분류 결과
    draft: str # 답장 초안

def classify(s: EmailState) -> dict:
    r = llm.invoke(PROMPT + s["email"])
    return {"category": r.content.strip()}

def write(s): return {"draft": s["category"]}

def route(s: EmailState) -> str:
    if s["category"] == SPAM: return END
    return "reply"

g = StateGraph(EmailState)
g.add_node("classify", classify)
g.add_node("reply", write)
g.add_edge(START, "classify")
g.add_conditional_edges("classify", route)
graph = g.compile()
print(*graph.stream(IN, stream_mode=MODE))
```



`StateGraph(EmailState)`로 빈 그래프를 열고,
`add_node`로 함수에 이름을 붙여 엮음.

`add_conditional_edges`의 판단 함수(`route`)가 다음 노드
이름이나 `END`를 직접 돌려줌. 등록되지 않은 이름을 돌려주면
실행이 깨짐.

`compile()`이 연결을 확정해 실행 가능한 그래프를 돌려줌.

4단계: 실행과 트레이스

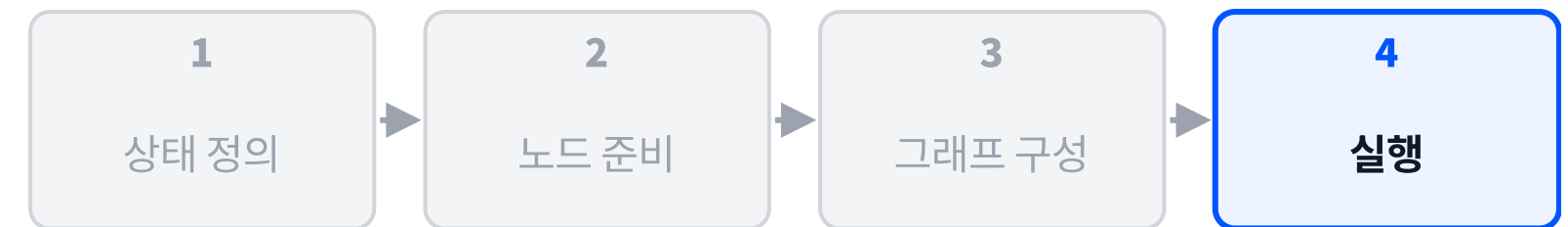
```
class EmailState(TypedDict):
    email: str # 메일 본문
    category: str # 분류 결과
    draft: str # 답장 초안

def classify(s: EmailState) -> dict:
    r = llm.invoke(PROMPT + s["email"])
    return {"category": r.content.strip()}

def write(s): return {"draft": s["category"]}

def route(s: EmailState) -> str:
    if s["category"] == SPAM: return END
    return "reply"

g = StateGraph(EmailState)
g.add_node("classify", classify)
g.add_node("reply", write)
g.add_edge(START, "classify")
g.add_conditional_edges("classify", route)
graph = g.compile()
print(*graph.stream(IN, stream_mode=MODE))
```



stream은 노드가 하나 끝날 때마다 바뀐 부분을 내보냄. 어느 노드가 무엇을 채웠는지가 화면에 그대로 찍힘.

```
{'classify': {'category': '환불'}}
{'reply': {'draft': '환불'}}
```

두 노드만 출력된 것이 조건부 엣지가 산 증거. 스팸이면 **classify**에서 바로 END로 빠져 **reply** 줄이 출력되지 않음.

손 루프와 그래프, 두 벌의 코드

손으로 짠 반복문

```
messages = [SystemMessage(SYSTEM),
             HumanMessage(question)]
for _ in range(max_turn):
    res = bound.invoke(messages)
    messages.append(res)
    if not res.tool_calls:
        break
    for call in res.tool_calls:
        out = TOOL_MAP[call["name"]].invoke(
            call["args"])
        messages.append(
            ToolMessage(str(out),
                        tool_call_id=call["id"]))
```

같은 일을 하는 그래프

```
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]

def llm_call(state: AgentState) -> dict:
    return {"messages":
            [bound.invoke(state["messages"])]}

g.add_node("llm_call", llm_call)
g.add_node("tools", ToolNode(TOOLS))
g.add_conditional_edges("llm_call",
                        should_continue,
                        {"tools": "tools", END: END})
g.add_edge("tools", "llm_call")
```

손 루프와 그래프의 대응 관계

손으로 짠 반복문	그래프	대응의 종류
<code>bound.invoke(messages)</code>	<code>llm_call</code> 노드	함수 → 노드
<code>TOOL_MAP[...].invoke(...)</code>	<code>ToolNode(TOOLS)</code>	도구 실행 → 노드
<code>if not res.tool_calls: break</code>	<code>should_continue</code> → <code>END</code>	종료 조건 → 조건부 엣지
<code>for _ in range(max_turn):</code>	<code>add_edge("tools", "llm_call")</code>	반복문 → 되돌아가는 엣지
<code>messages.append(...)</code>	<code>Annotated[list, add_messages]</code>	수동 누적 → 상태 갱신 규칙

같은 질문·같은 도구로 두 벌을 돌리면 도구 호출 트레이스가 동일함

```
[('faq_lookup', {'topic': '환불'}), ('refund_amount', {'price': 30000, 'days_before': 1})]
```

모델이 쓰는 답변 문장은 호출마다 조금씩 달라짐. 두 벌이 같은 일을 한다는 판정은 도구 호출 트레이스로 함.

실습합시다

챕터 요약

- ✓ 상태는 노드들이 함께 읽고 쓰는 상태 컨테이너, 노드는 상태를 받아 바뀐 부분만 돌려주는 함수, 엣지는 노드 사이의 순서를 정하는 선
- ✓ 그래프는 네 단계로 세움: 상태 정의, 노드 준비, 그래프 구성, 실행
- ✓ 조건부 엣지(라우팅)는 판단 함수가 돌려준 이름으로 다음 노드를 고르며, 그 이름의 노드가 그래프에 등록돼 있어야 함
- ✓ 손으로 짠 반복문은 그래프에서 노드로 되돌아가는 엣지가 되고, 두 벌의 도구 호출 트레이스는 동일함

Agentic 워크플로우

체크포인트와 사람 개입 (Human-in-the-loop, HITL)

중단과 재개 / Checkpoint / 사람 개입

학습 목표

- 체크포인트(Checkpointer)와 thread_id로 그래프 실행을 중단 지점부터 재개할 수 있습니다.
- `interrupt()` 로 노드 실행을 멈춰 사람 입력을 받고 `Command(resume=)` 으로 재개하는 흐름을 승인과 수정 두 패턴으로 구현할 수 있습니다.
- 개입 지점을 어디에 둘지 비용과 위험을 기준으로 고르고, 그 근거를 설명할 수 있습니다.
- 같은 thread_id로 이어지는 대화에서 State.messages가 컨텍스트가 됨을 설명하고, 컨텍스트 관리 노드로 오래된 턴을 요약해 되돌려 쓸 수 있습니다.

✓ 상태·노드·조건부 엣지

▶ 체크포인트와 사람 개입

서브그래프 모듈화

추상화 스펙트럼과 선택 기준

MCP 구조와 연결

커스텀 MCP 서버

멀티모달 입출력

이미지 생성 파이프라인

7번째 처리 중 종료된 1차 프로세스

```
=== 1차 프로세스: 실행 ===  
... [처리 1/10]~[처리 6/10] LLM 호출 6회 ...  
Traceback (most recent call last):  
...  
RuntimeError: [강제 예외] 7번째 처리 중 프로세스 사망  
##### exit=1 #####
```

종료 코드 1. 파이썬 프로세스가 여기서 끝났고, 1~6번의 LLM 호출 비용은 이미 지불됨.

체크포인트(Checkpoint)를 인수한 2차 프로세스

=== 2차 프로세스: 작은 프로세스의 체크포인트 인수 ===

[인수한 상태] **idx = 6**

[인수한 상태] **results** = ['환불', '배송', '기타', '기타', '스팸', '기타']

[다음 실행 노드] **next** = ('process_one',)

=== 재개 (invoke(None, config)) ===

[처리 7/10] LLM 호출 -> 교환 신청은 어디서 하나요?...

[처리 8/10] LLM 호출 -> 결제가 두 번 됐습니다....

[처리 9/10] LLM 호출 -> 배송지 변경 가능한가요?...

[처리 10/10] LLM 호출 -> 영수증 재발급 부탁드립니다....

[최종] 처리 건수 = 10

재개한 프로세스의 첫 줄이 [처리 7/10]. 1~6번은 이 프로세스에서 한 번도 호출되지 않음(LLM 재호출 0회).

문제상황과 목표

문제

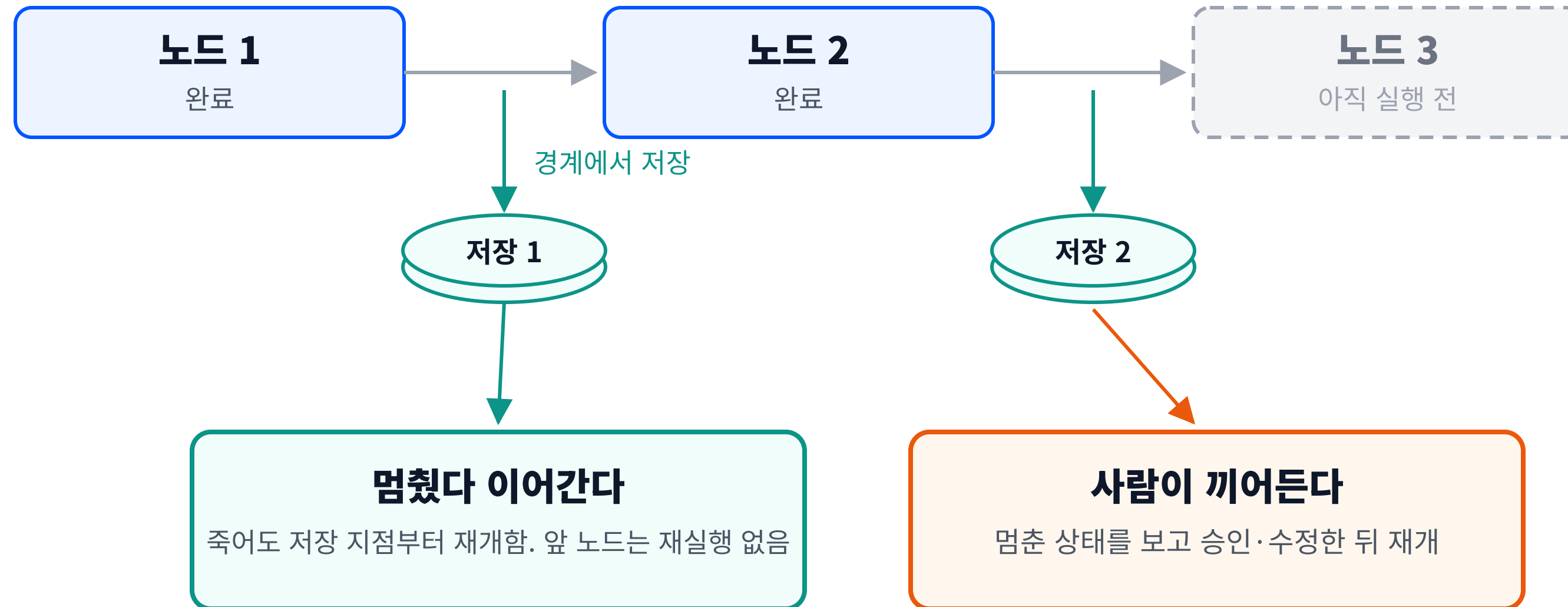
쇼핑몰 고객센터예요. 밤새 쌓인 메일 열 통을 처리하다 중간에 프로그램이 꺼지면 처음부터 다시 돌아요. 발송 전에 사람이 한 번 확인하고 싶어요.

목표

한 통 끝날 때마다 진행 상황을 저장해 두어, 꺼져도 그 지점부터 이어 가고 사람이 중간에 확인하고 고친 뒤 이어 갈 수 있게 만들어봅시다.

노드 경계의 저장과 거기서 나누는 두 경로

실행: 노드 경계마다 상태가 저장된다



저장 지점이 있어야 재개도 개입도 성립함.

각 단계별 요구사항

- **노드 1·노드 2·노드 3**: 밤새 쌓인 메일을 분류하고, 답장 초안을 쓰고, 발송하는 메일 처리의 각 단계. 노드는 상태를 받아 자기 단계의 일을 하고, 바뀐 키를 상태에 돌려준다
- **저장 1·저장 2**: 노드가 끝나는 경계마다, 그 시점의 상태 전체를 thread_id 아래 체크포인트로 남긴다
- **멈췄다 이어간다**: 처리 중 프로그램이 꺼져도, 같은 thread_id로 다시 부르면 마지막 저장 지점부터 재개한다. 이미 끝난 노드는 다시 돌리지 않는다
- **사람이 끼어든다**: 발송처럼 사람 확인이 필요한 위치에서 실행을 멈추고, 사람이 멈춘 상태를 승인하거나 고친 뒤 그 상태로 이어 간다

체크포인트(Checkpoint)는 **노드 실행이 끝나는 경계마다 그 시점의 상태 전체를 저장한 기록.**

노드를 잘게 나눌수록 저장이 잦아지고, 잘못됐을 때 다시 해야 하는 일이 줄어듦.

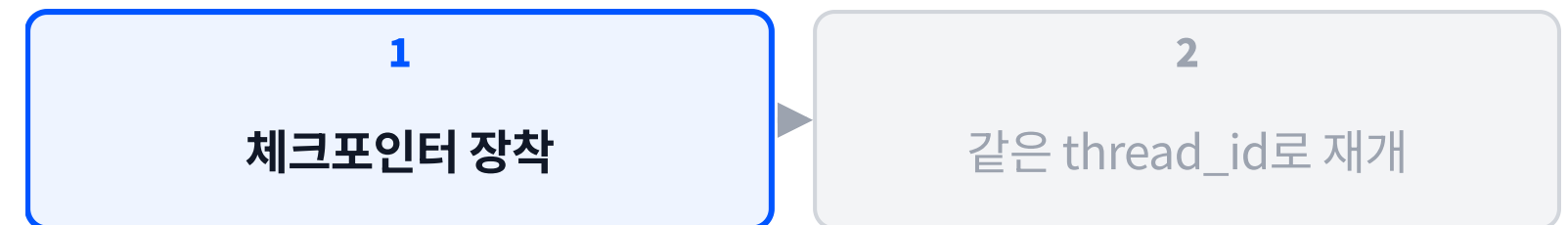
“Smaller nodes mean more frequent checkpoints, which means less work to repeat if something goes wrong.”

1단계: 체크포인트 장착

```
conn = sqlite3.connect(
    DB_PATH, check_same_thread=False)
saver = SqliteSaver(conn)
saver.setup()

g = StateGraph(BatchState)
g.add_node("process_one", process_one)
g.add_edge(START, "process_one")
g.add_conditional_edges(
    "process_one", more_left,
    {"process_one": "process_one", END: END})
graph = g.compile(checkpointer=saver)

THREAD = {"configurable":
    {"thread_id": "batch-sqlite-001"}}
snap = graph.get_state(THREAD)
final = graph.invoke(None, THREAD)
```



compile(checkpointer=...)가 저장을 컴. 연결과 노드는 한 글자도 바뀌지 않음.

상태를 프로세스 메모리에만 두는 체크포인트라면, 프로세스가 죽을 때 상태도 함께 사라짐.

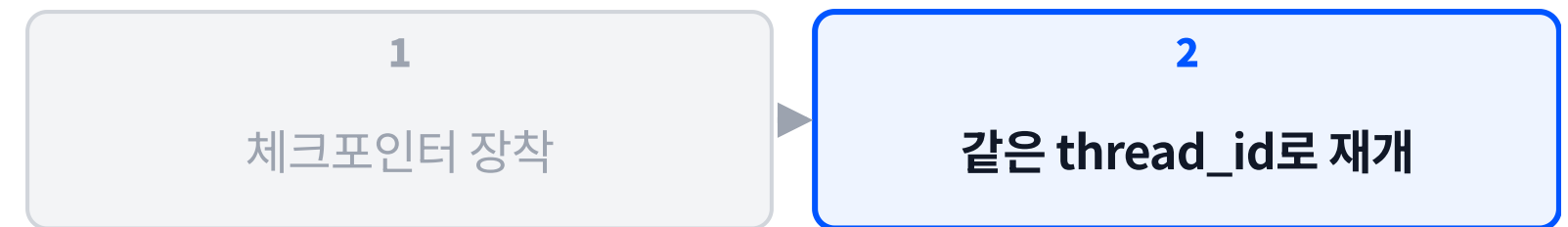
SqliteSaver는 상태를 파일에 둬. 프로세스가 죽어도 다음 프로세스가 그 파일에서 이어받음.

2단계: 같은 thread_id로 재개

```
conn = sqlite3.connect(
    DB_PATH, check_same_thread=False)
saver = SqliteSaver(conn)
saver.setup()

g = StateGraph(BatchState)
g.add_node("process_one", process_one)
g.add_edge(START, "process_one")
g.add_conditional_edges(
    "process_one", more_left,
    {"process_one": "process_one", END: END})
graph = g.compile(checkpointer=saver)

THREAD = {"configurable":
    {"thread_id": "batch-sqlite-001"}}
snap = graph.get_state(THREAD)
final = graph.invoke(None, THREAD)
```



thread_id가 저장된 실행을 가리키는 이름. 두 프로세스를 잇는 유일한 끈.

get_state는 어떤 노드도 실행하지 않고 저장된 상태를 읽음.
여기서 나온 값이 **idx = 6**.

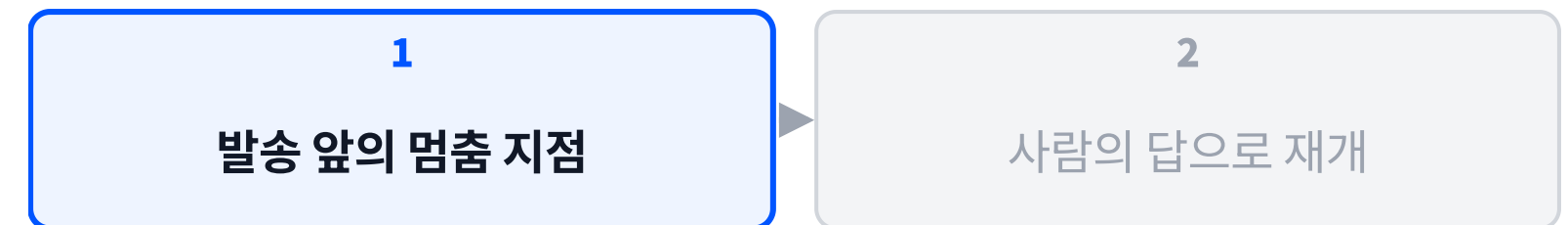
입력 위치의 **None**이 "새로 시작이 아니라 이어서"의 표식.

interrupt()는 노드 실행을 그 자리에서 멈추고, 사람이 준 값을 그 호출의 반환값으로 받아 이어 가는 함수. 사람 개입(Human-in-the-loop, HITL)이 이 함수로 구현됨.

멈춤과 재개가 저장된 상태 위에서 일어나므로 체크포인트가 없으면 동작하지 않음.
개입 지점은 비용과 위험이 큰 동작 앞에 둬.

1단계: 발송 앞의 멈춤 지점

```
def human_review(state: ReviewState) -> dict:
    answer = interrupt(
        {"질문": "이 초안을 발송할까요?",
         "초안": state["draft"]})
    if isinstance(answer, dict):
        return {"draft": answer["edit"],
                "decision": "수정 후 승인"}
    return {"decision": str(answer)}
g.add_node("human_review", human_review)
g.add_edge("draft", "human_review")
g.add_edge("human_review", "send")
graph = g.compile(checkpointer=MemorySaver())
ok = {"configurable": {"thread_id": "t-ok"}}
graph.invoke({"email": EMAIL}, ok)
graph.invoke(Command(resume="승인"), ok)
ed = {"configurable": {"thread_id": "t-ed"}}
graph.invoke({"email": EMAIL}, ed)
graph.invoke(Command(resume=EDIT), ed)
```



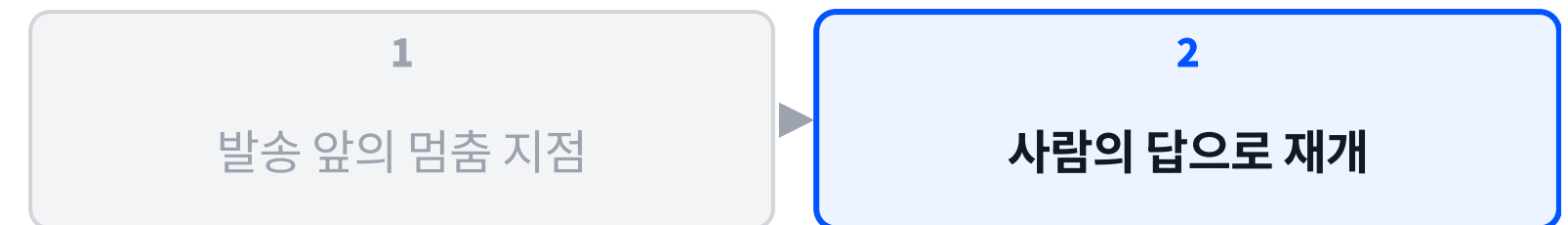
interrupt(payload)의 payload가 사람 쪽으로 그대로 나감. 초안을 보여 주는 통로가 이곳.

멈춘 그래프의 **next**는 ('human_review',). 발송 노드는 아직 실행되지 않았음.

그래프가 멈춰 있는 동안 실행 중인 프로세스는 없음. 몇 시간을 기다려도 점유하는 자리가 없음.

2단계: 사람의 답으로 재개

```
def human_review(state: ReviewState) -> dict:
    answer = interrupt(
        {"질문": "이 초안을 발송할까요?",
         "초안": state["draft"]})
    if isinstance(answer, dict):
        return {"draft": answer["edit"],
                "decision": "수정 후 승인"}
    return {"decision": str(answer)}
g.add_node("human_review", human_review)
g.add_edge("draft", "human_review")
g.add_edge("human_review", "send")
graph = g.compile(checkpointer=MemorySaver())
ok = {"configurable": {"thread_id": "t-ok"}}
graph.invoke({"email": EMAIL}, ok)
graph.invoke(Command(resume="승인"), ok)
ed = {"configurable": {"thread_id": "t-ed"}}
graph.invoke({"email": EMAIL}, ed)
graph.invoke(Command(resume=EDIT), ed)
```



`Command(resume=값)`의 값이 `interrupt()`의 반환값으로 들어옴.

승인은 문자열 그대로 통과해 초안이 발송됨. 수정은 노드가 `edit` 키를 읽어 초안을 갈아 끼운 뒤 발송함.

값의 형태를 해석하는 것은 노드. 그래프가 정해 주지 않음.

대화 메모리는 체크포인트가 `thread_id`마다 보존하는 `State.messages` 그 자체. 새 장치가 아니라 재개 장치의 두 번째 쓰임.

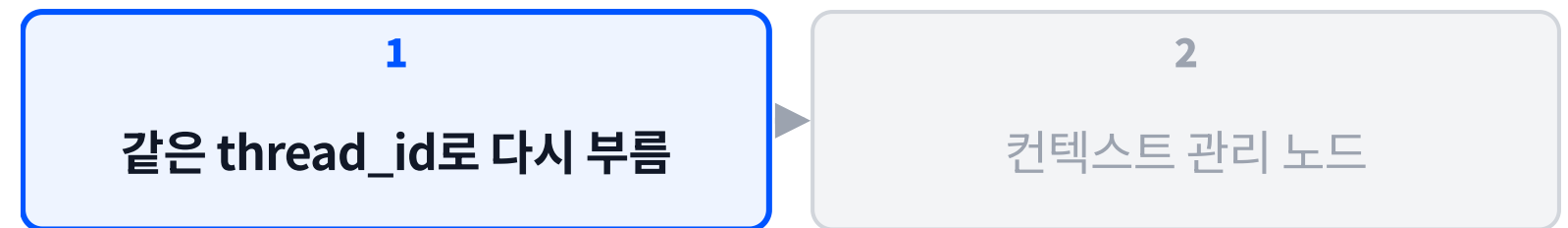
`State.messages`가 곧 모델에 들어가는 컨텍스트. 턴이 쌓이면 컨텍스트도 그만큼 자람.

장기 기억(long-term memory)은 외부 저장소(Store)의 몫이고, 여기서는 단기 기억(short-term memory, State)만 다룸.

“Short-term memory (thread-level persistence) enables agents to track multi-turn conversations.”

1단계: 같은 thread_id로 다시 부름

```
graph = build_v1()
cfg = {"configurable":
      {"thread_id": "user-a"}}
ask = lambda q: graph.invoke(
      {"messages": [HumanMessage(q)]}, cfg)
ask("제 이름은 김민준입니다.")
out = ask("제 이름이 뭐라고 했죠?")
print(len(out["messages"]),
      out["messages"][-1].content)
```



build_v1()은 노드가 **model** 하나인 그래프를 **compile(checkpointer=MemorySaver())**로 컴파일함. 메모리를 위해 더한 코드는 없음.

실행 결과: **4 김민준이라고 하셨습니다.** 두 번째 호출의 messages가 4건(사람 2·답 2). 첫 턴의 이름이 두 번째 답에 실림.

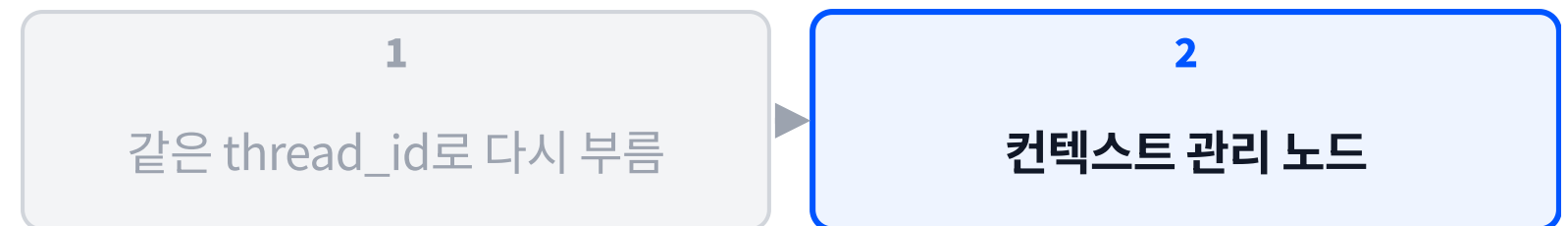
다른 thread_id(**user-b**)로 같은 질문을 하면 "아직 이름을 말씀해 주시지 않았어요". 기억의 단위는 thread_id.

2단계: 컨텍스트 관리 노드

```
KEEP = 4

def manage(state: State) -> dict:
    msgs = state["messages"]
    if len(msgs) <= KEEP: return {}
    old = msgs[:-KEEP]
    summary = llm.invoke(
        [SystemMessage(SUMMARY_RULE)] + old
    ).content
    gone = [RemoveMessage(id=m.id)
            for m in old]
    note = SystemMessage("[요약] " + summary)
    return {"messages": gone + [note]}

g = StateGraph(State)
g.add_node("manage", manage)
g.add_node("model", call_model)
g.add_edge(START, "manage")
g.add_edge("manage", "model")
g.add_edge("model", END)
graph = g.compile(checkpointer=MemorySaver())
```



원칙은 코드에 미리 적혀 있음. **KEEP**이 남길 건수, **SUMMARY_RULE**이 줄이는 방식. 노드는 판단하지 않고 그 원칙대로 줄여 되돌려 씀.

RemoveMessage(id=)가 **add_messages** 리듀서(reducer)에 지우라는 표식으로 들어가고, 요약 SystemMessage 한 건이 그 위치를 채움. messages는 6건에서 더 늘지 않음.

잘라 내면 사실이 사라지고, 요약하면 남음. 같은 4턴을 요약 없이 최근 4건만 남기면(트리밍(trimming), **trim_messages**) 4턴째 답이 "이름: 미정". 요약해 되돌려 쓰면 "민준님, 예산은 3만 원 이하, 원하는 색상은 검정색".

실습합시다

챕터 요약

- ✓ 노드 실행이 끝나는 경계마다 그 시점의 상태 전체가 저장되고, 저장 지점이 중단과 재개의 단위가 됨
- ✓ 실행을 멈췄던 thread_id로 다시 부르면 저장 지점부터 이어 감. 입력 위치의 None이 이어서 하라는 표식
- ✓ 체크포인트를 파일 DB로 바꾸면 프로세스가 죽어도 다음 프로세스가 상태를 이어받음. 코드 차이는 compile의 인자뿐
- ✓ interrupt()가 노드 실행을 멈추고, Command(resume=값)가 사람의 답을 담아 재개함. 값의 형태는 노드가 해석함
- ✓ 대화 메모리: 같은 thread_id로 다시 부르면 State.messages가 이어져 앞 턴을 기억하고, 컨텍스트 관리(context management) 노드가 미리 적힌 원칙대로 오래된 턴을 요약(summarization)해 되돌려 씀
- ✓ 사람 개입(HITL) 지점은 비용과 위험이 큰 동작 앞에 둠

Agentic 워크플로우

서브그래프 모듈화

커진 그래프 / 서브그래프 / 스키마 공유·비공유 / 동적 확장

학습 목표

- 서브그래프(Subgraph)를 부모 그래프에 엮는 두 방식을 상태 스키마의 공유 여부를 기준으로 구별할 수 있습니다.
- 상태를 공유하는 범위를 바탕으로 그래프의 경계를 나눌 수 있습니다.

✓ 상태·노드·조건부 엣지

✓ 체크포인트와 사람 개입

▶ 서브그래프 모듈화

추상화 스펙트럼과 선택 기준

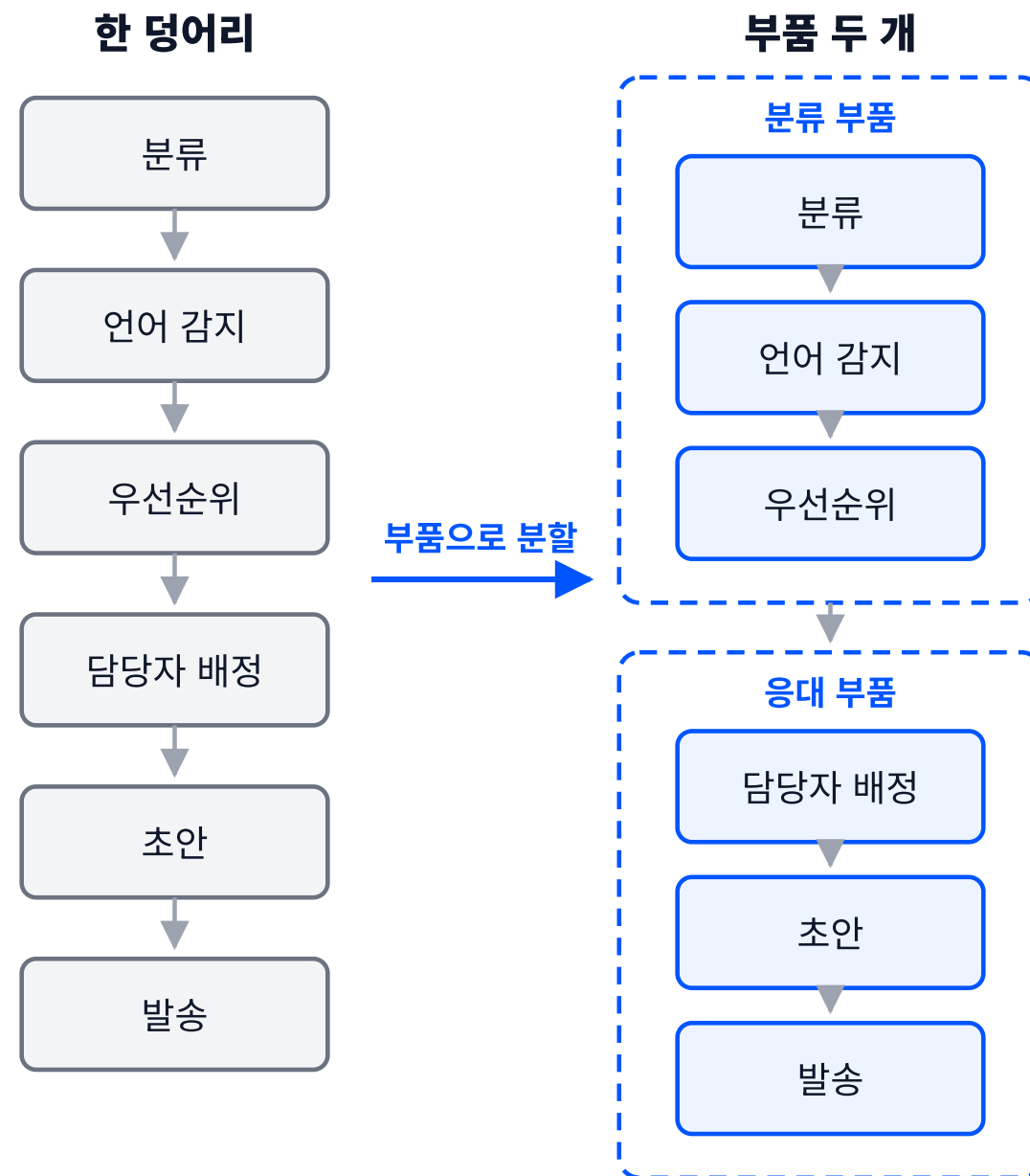
MCP 구조와 연결

커스텀 MCP 서버

멀티모달 입출력

이미지 생성 파이프라인

한 덩어리로 커진 그래프

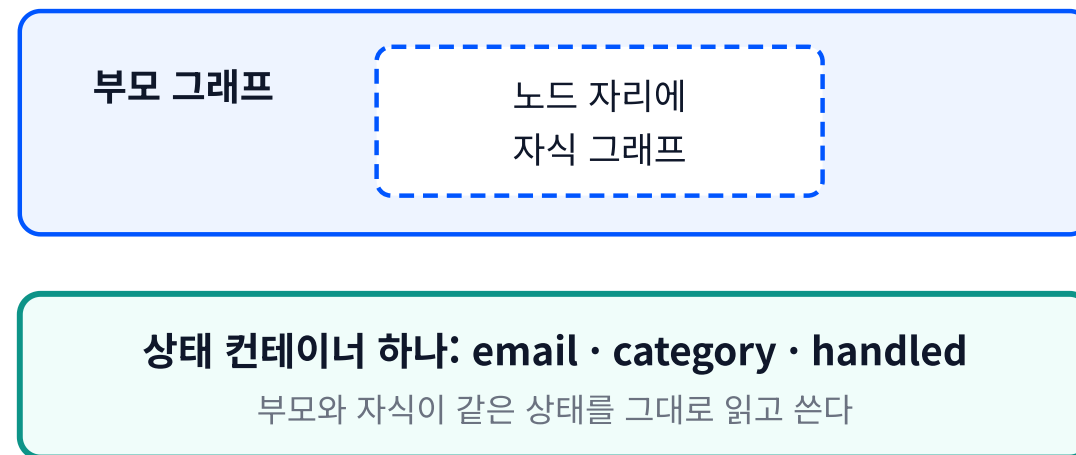


- 요구가 하나씩 늘어날 때마다 노드 함수와 **add_edge** 호출이 같은 파일에 함께 쌓임
- 분류 부분만 따로 돌려 보려면 그래프 전체를 실행해야 함
- 같은 분류 절차를 다른 그래프에서 쓰려면 노드 함수와 연결을 다시 옮겨 적어야 함

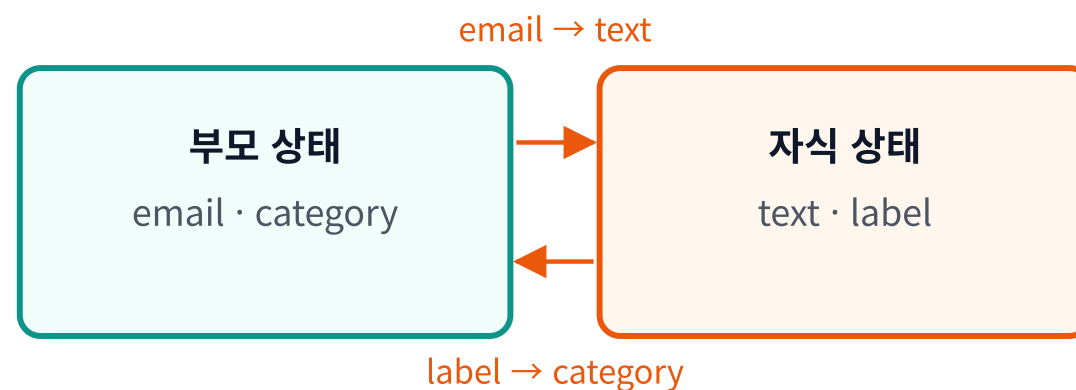
그래프는 노드로만 채우는 것이 아니라, 그래프 자체를 노드로 등록할 수 있음

서브그래프(Subgraph)

스키마 공유: 상태가 하나



스키마 비공유: 상태가 둘, 번역이 필요



번역을 맡는 것이 자식을 감싼 함수. 부모 키를 자식 키로 바꿔 넣고, 결과를 되받음

다른 그래프의 노드로 쓰이는, 따로 컴파일된 그래프

- 부모 그래프(parent)가 자식 그래프(child)를 노드로 실행하고, 자식의 실행이 끝나면 부모의 다음 엣지로 이어짐
- 자식도 그 자체로 완결된 그래프. 따로 컴파일해 단독으로 실행할 수 있음
- 부모와 자식이 **같은 상태 스키마**를 쓰는 경우와 **서로 다른 상태 스키마**를 쓰는 경우에 엮는 방법이 나뉨

방식 A 1단계: 서브그래프 구성과 컴파일

```
def build_shared_sub():
    g = StateGraph(SharedState)
    g.add_node("detect", detect_lang)
    g.add_node("classify", sub_classify)
    g.add_edge(START, "detect")
    g.add_edge("detect", "classify")
    g.add_edge("classify", END)
    return g.compile()

def build_parent_shared():
    g = StateGraph(SharedState)
    g.add_node("classifier",
               build_shared_sub())
    g.add_node("handle", handle)
    g.add_edge(START, "classifier")
    g.add_edge("classifier", "handle")
    g.add_edge("handle", END)
    return g.compile()
```

1

서브그래프 구성과 컴파일

2

부모 그래프에 노드로 엮기

SharedState 부모가 쓰는 상태 스키마 그대로를 자식도 씀.
email·**category**·**handled** 세 키가 부모와 자식의 공용 상태.

detect·**classify** 자식 안의 노드 둘. 연결 방식은 부모
그래프와 완전히 같음.

g.compile() 자식도 단독으로 **invoke**할 수 있는 완성된
그래프가 됨.

방식 A 2단계: 부모 그래프에 노드로 엮기

```
def build_shared_sub():
    g = StateGraph(SharedState)
    g.add_node("detect", detect_lang)
    g.add_node("classify", sub_classify)
    g.add_edge(START, "detect")
    g.add_edge("detect", "classify")
    g.add_edge("classify", END)
    return g.compile()
```

```
def build_parent_shared():
    g = StateGraph(SharedState)
    g.add_node("classifier",
               build_shared_sub())
    g.add_node("handle", handle)
    g.add_edge(START, "classifier")
    g.add_edge("classifier", "handle")
    g.add_edge("handle", END)
    return g.compile()
```

1

서브그래프 구성과 컴파일

2

부모 그래프에 노드로 엮기

add_node 두 번째 인자가 파이썬 함수가 아니라 컴파일된 그래프. 그래프를 노드로 그대로 등록하는 것이 이 방식의 전부.

"classifier" 부모가 붙이는 노드 이름. 자식 안의 노드 이름 (**detect·classify**)과는 무관함.

부모가 자식의 상태를 그대로 읽고 씀. 키를 옮겨 담는 코드가 없음.

방식 B 1단계: 부모 키를 자식 키로 번역

```
class SubState(TypedDict): # 키 이름이 다름
    text: str
    label: str
PRIVATE_SUB = build_private_sub()

def call_sub(state: SharedState) -> dict:
    out = PRIVATE_SUB.invoke(
        {"text": state["email"]})
    return {"category": out["label"]}

def build_parent_private():
    g = StateGraph(SharedState)
    g.add_node("classifier", call_sub)
    g.add_node("handle", handle)
    g.add_edge(START, "classifier")
    g.add_edge("classifier", "handle")
    g.add_edge("handle", END)
    return g.compile()
```

1

부모 키를 자식 키로 번역

2

감싼 함수를 노드로 엮기

SubState 자식이 쓰는 상태 스키마. 키 이름이 **text·label**이라 부모의 **email·category**와 맞지 않음.

call_sub 부모 상태를 자식 상태로 바꿔 넣고, 결과를 부모 키로 되돌려 받는 함수. **email**을 **text**로 넣고 **label**을 **category**로 되받는 대목이 이 방식의 핵심.

자식 그래프 자체를 세우는 절차는 스키마 공유 방식과 같음. 다른 것은 상태 스키마와, 그래서 필요해진 번역뿐.

방식 B 2단계: 래퍼 함수를 노드로 등록하기

```
class SubState(TypedDict):    # 키 이름이 다름
    text: str
    label: str
PRIVATE_SUB = build_private_sub()

def call_sub(state: SharedState) -> dict:
    out = PRIVATE_SUB.invoke(
        {"text": state["email"]})
    return {"category": out["label"]}

def build_parent_private():
    g = StateGraph(SharedState)
    g.add_node("classifier", call_sub)
    g.add_node("handle", handle)
    g.add_edge(START, "classifier")
    g.add_edge("classifier", "handle")
    g.add_edge("handle", END)
    return g.compile()
```

1

부모 키를 자식 키로 번역

2

감싼 함수를 노드로 엮기

add_node 노드로 등록되는 것이 서브그래프가 아니라 그것을 감싼 래퍼 함수. 연결의 나머지는 방식 A와 같음.

=== 방식 A: 상태 스키마 공유 (노드로 직접) ===

결과: category='배송' handled='배송 담당자에게 배정'

=== 방식 B: 스키마 비공유 (변환 함수로 감싸기) ===

결과: category='배송' handled='배송 담당자에게 배정'

동적 그래프 확장: Send, 병렬화(parallelization, fan-out)

```
class FanState(TypedDict):
    emails: list
    urgent: list
    replies: Annotated[list, operator.add] # 병렬 노드가 동시에 쓴다 → 리듀서 필수

def fan_out(state: FanState):
    """정적 배선으로는 그릴 수 없는 자리. 노드 '개수'가 여기서 정해진다."""
    return [Send("handle_one", {"email": e}) for e in state["urgent"]]

g.add_conditional_edges("triage", fan_out, ["handle_one"]) # 반환값이 Send 목록
```

```
=== 입력 3건 (그래프 코드는 그대로) ===
[펼쳐진 노드 수] 2
=== 입력 5건 (그래프 코드는 그대로) ===
[펼쳐진 노드 수] 4
```

- 조건부 엣지 함수가 **Send 목록**을 돌려주면, 목록 길이만큼 노드 인스턴스가 실행 시점에 팬아웃됨
- `add_node` 는 한 번뿐인데 실행 인스턴스는 입력마다 다름. 이 수는 미리 그려 둘 수 없음
- 팬아웃된 노드들이 같은 키에 동시에 쓰므로 리듀서 없는 키는 그 자리에서 예외가 발생함

실습합시다

챕터 요약

- ✓ 서브그래프(Subgraph)는 다른 그래프의 노드로 쓰이는, 따로 컴파일된 그래프
- ✓ 부모와 자식이 상태 스키마를 공유하면 컴파일한 서브그래프를 노드로 그대로 등록하고, 공유하지 않으면 부모 키를 자식 키로 번역하는 함수를 노드로 등록함. 두 방식의 결과는 같음
- ✓ 그래프를 나눌 경계는 상태 공유 범위로 정함. 부모 상태를 그대로 읽고 써야 하면 공유하고, 다른 그래프에서도 쓸 부품이면 상태를 분리함
- ✓ 조건부 엣지가 Send 목록을 돌려주면 노드 인스턴스가 실행 시점에 팬아웃됨. 미리 연결해 둘 수 없는 개수를 다루는 곳

Agentic 워크플로우

추상화 스펙트럼과 선택 기준

스펙트럼 / 선택의 기준 / 선택 연습

학습 목표

- 직접 구현 루프, 랭그래프(LangGraph), 고수준 프레임워크, 노코드 도구가 통제력과 편의성 사이에서 어디에 위치하는지 설명할 수 있습니다.
- 감사 로그, 규제, 마감 같은 요구사항이 주어졌을 때 직접 구현 루프 · 랭그래프 · 고수준 프레임워크 · 노코드 도구 가운데 무엇을 쓸지 고르고, 그 근거를 요구사항과 이어 설명할 수 있습니다.

✓ 상태 · 노드 · 조건부 엣지

✓ 체크포인트와 사람 개입

✓ 서브그래프 모듈화

▶ 추상화 스펙트럼과 선택 기준

MCP 구조와 연결

커스텀 MCP 서버

멀티모달 입출력

이미지 생성 파이프라인

도구 선택 질문



수강생

랭체인 말고 CrewAI 쓰면 안 되나요? 역할만 적어 두면 알아서 돌아간다던데요.

됩니다. 다만 추적 지점은 프레임워크가 정한 곳에서만 잡히는데, 답변 근거를 건별로 남겨야 하는 규정 안내 봇도 그 지점으로 충분할까요?

그건 안에서 무슨 일이 일어나는지 볼 수 있어야 할 것 같은데요.



강사

추상화 수준의 스펙트럼



같은 에이전트 루프를 어느 높이에서 다루는가에 따라 네 단계가 나뉨.

**통제력은 실행 중 어느 지점에 손을 댈 수 있는가이고,
편의성은 그 지점을 감춘 대가로 얻는 속도.**

둘은 같이 커지지 않음. 한쪽을 얻으면 다른 쪽을 내주는 관계(trade-off).

네 위치의 비교

기준	직접 구현 루프	랭그래프	고수준 프레임워크 (CrewAI · AutoGen)	노코드 도구
루프를 누가 쓰는가	while문 직접 작성	노드와 엣지 직접 연결	프레임워크 내장	플랫폼 내장
손댈 수 있는 지점	코드 전 구간	상태·노드 경계· 분기	역할과 과업 선언 범위	화면이 제공하는 설정
세우는 속도	가장 느림	중간	빠름	가장 빠름
틀 밖으로 나가기	제한 없음	제한 적음	프레임워크가 정한 데까지	어려움

우열이 아니라 위치. 무엇을 남기고 어디서 멈춰야 하는가가 위치를 정함.

실무 단축형 create_agent

```
from langchain.agents import create_agent
from langchain.chat_models import (
    init_chat_model)

llm = init_chat_model(
    "openai/gpt-5.6-luna",
    model_provider="litellm",
)

agent = create_agent(
    llm,
    tools=[faq_lookup, refund_amount],
    system_prompt=SYSTEM,
)

agent.invoke({"messages": [
    {"role": "user", "content": inquiry}]})
```

같은 루프가 한 번의 호출로 줄어든다

직접 구성할 때	단축형
상태 정의	메시지 이력 기본값
노드 연결	내장 루프
조건부 엣지	내장 종료 조건
개입 지점 지정	별도 설정

줄어든 만큼이 감춰진 것. 상태·분기·개입 지점을 직접 정해야 하는 요구사항은 감춰진 내부를 다시 열어야 함.

사내 규정 안내 봇

요구사항

"근거와 이력을 남겨야 한다"

어떤 규정 문서를 근거로 답했는지 문의 건마다 기록함.

발송 전 담당자 승인을 한 번 받고, 승인 이력은 감사 대상이 됨.

위치 판정

랭그래프

기록을 남길 지점과 멈출 지점을 직접 지정할 수 있어야 함.

노드 경계마다 상태가 남고, 승인은 조건부 분기 한 개로 가능함.

직접 구현 루프

랭그래프

고수준 프레임워크

노코드 도구

사내 행사 공지 자동화

요구사항

"이번 주 안에 돌아가야 한다"

행사 정보를 받아 공지 초안을 쓰고 사내 메신저로 보냄.

흐름은 매주 같고 분기가 없으며, 잘못 나가면 다시 보내면 됨.

위치 판정

노코드 도구

남길 것도 멈출 곳도 없고 마감이 짧음.

화면에서 블록을 잇는 것만으로 요구사항이 전부 채워짐.

직접 구현 루프

랭그래프

고수준 프레임워크

노코드 도구

실습합시다

챕터 요약

- ✓ 문제 상황의 성격(통제력·편의성·속도)에 따라 직접 구현 루프, 랭그래프, 고수준 프레임워크, 노코드 도구 가운데 더 적절한 것을 골라야 함
- ✓ 통제력은 실행 중 손댈 수 있는 지점의 범위이고, 편의성은 그 지점을 감춘 대가로 얻는 속도
- ✓ 감사 로그나 승인처럼 남길 지점과 멈출 지점이 정해진 요구사항은 내부가 보이는 쪽을 요구함
- ✓ create_agent는 에이전트 루프를 줄인 실무 단축형이며, 줄어든 만큼 상태·분기·개입 지점이 기본값으로 감춰짐

Agentic 워크플로우

MCP 구조와 연결

M×N 문제 / MCP 구조 / 서버 연결

학습 목표

- MCP(Model Context Protocol)의 서버(Server)와 클라이언트(Client) 구조, 전송 방식(stdio · Streamable HTTP)의 차이를 설명하고, stdio 연결 코드를 바탕으로 구조를 설명할 수 있습니다.
 - 기성 MCP 서버 하나를 에이전트에 연결해 (MultiServerMCPClient → get_tools → bind_tools) 도구 호출을 성립시킬 수 있습니다.
- ✓ 상태·노드·조건부 엣지
 - ✓ 체크포인트와 사람 개입
 - ✓ 서브그래프 모듈화
 - ✓ 추상화 스펙트럼과 선택 기준
 - ▶ **MCP 구조와 연결**
 - 커스텀 MCP 서버
 - 멀티모달 입출력
 - 이미지 생성 파이프라인

문제 상황: 규격 없이 도구마다 다시 짜는 연결 코드

```
integrations/  
weather_provider_a.py 날씨 조회 · A 호출 규격  
weather_provider_b.py 날씨 조회 · B 호출 규격  
sqldb_provider_a.py 사내 DB 조회 · A 호출 규격  
sqldb_provider_b.py 사내 DB 조회 · B 호출 규격  
drive_provider_a.py 파일 검색 · A 호출 규격  
drive_provider_b.py 파일 검색 · B 호출 규격
```

도구 하나를 붙일 때마다 연결 코드 하나.

도구	모델 계열	연결 코드
3	2	6벌
4	2	8벌
4	3	12벌

도구 M개와 모델 N개를 각각 이어 붙이면 연결 코드는 $M \times N$ 벌.

도구 하나가 늘면 N벌이 늘고, 모델을 갈아 끼우면 전부 다시 씀.

문제상황과 목표

문제

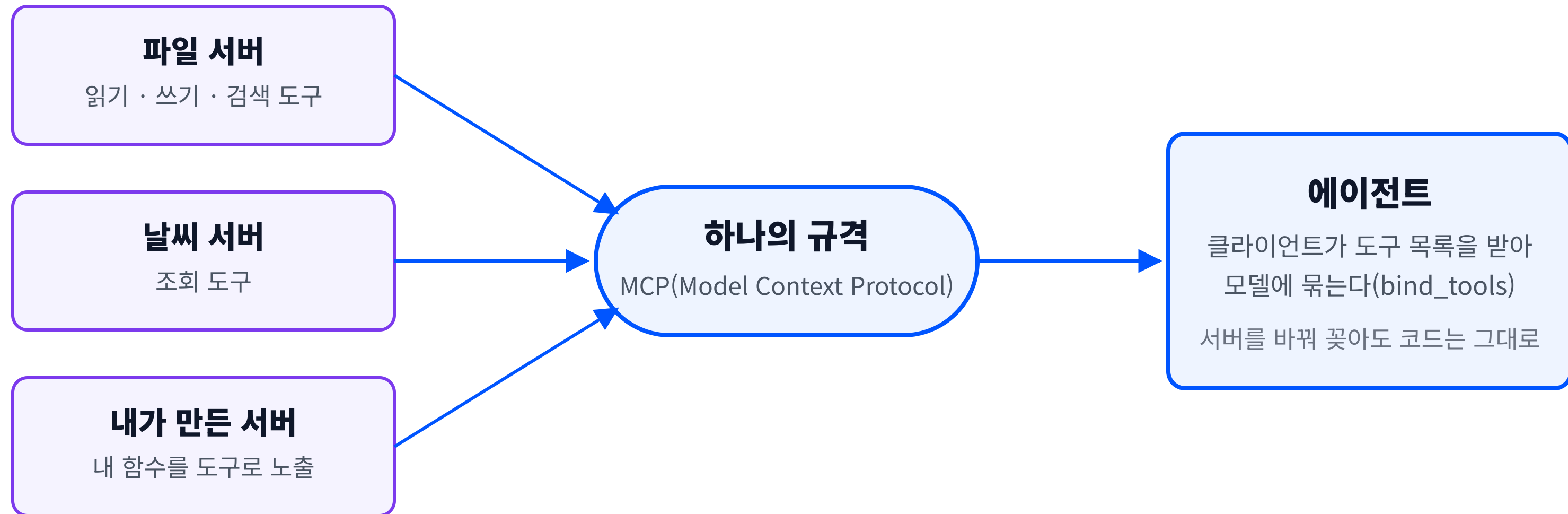
사내 개발팀이에요. 계산기·상점 조회처럼 쓸 도구가 늘 때마다 프로그램마다 연결 방법을 따로 짜야 해요.

목표

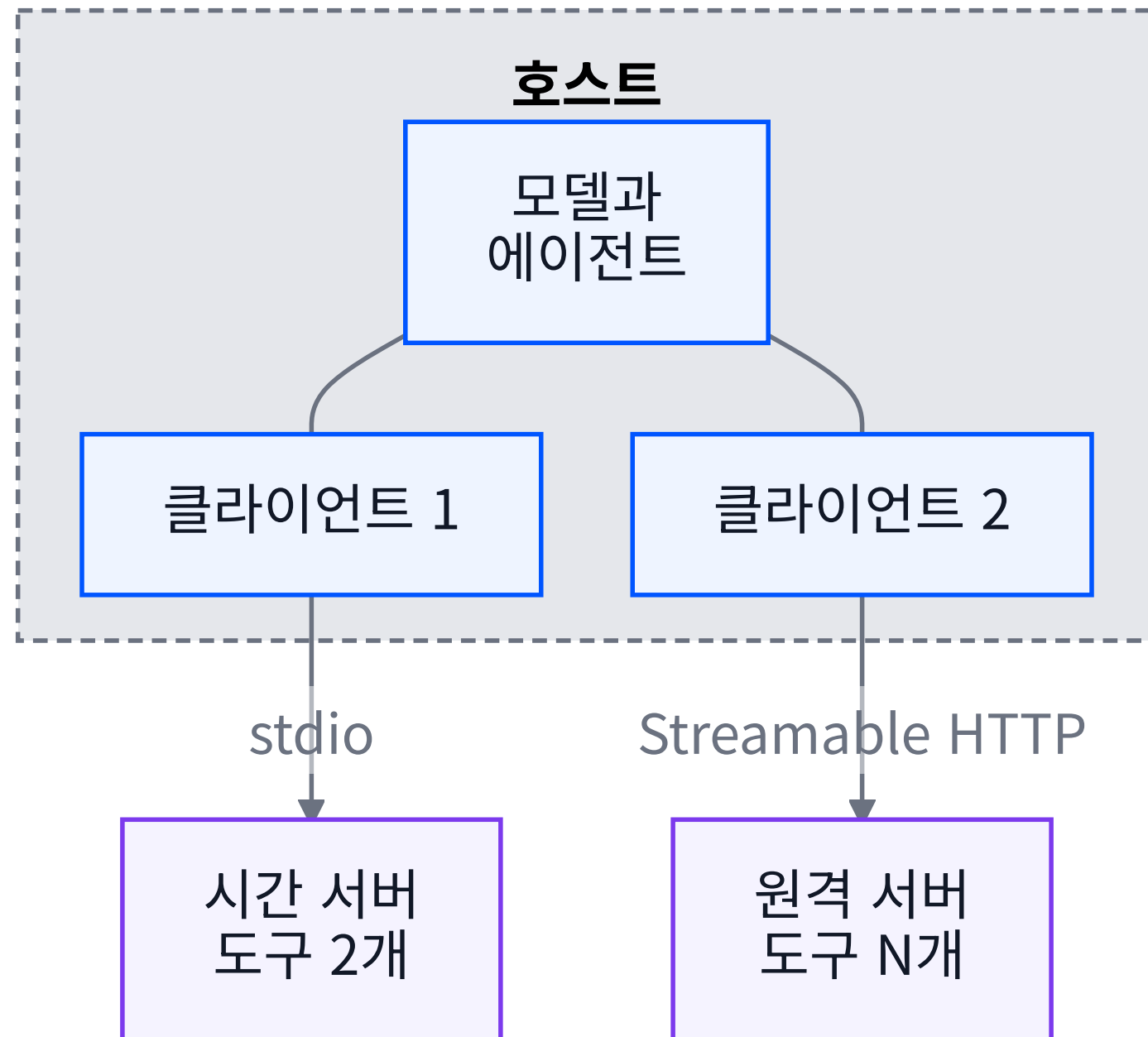
도구를 한 가지 규격으로 연결해 두면 어떤 프로그램에서든 같은 방법으로 찾아 쓰게 되는 구조를 봅시다.

하나의 규격으로 모이는 연결

MCP 서버: 도구·데이터를 내놓는 쪽



서버, 클라이언트, 호스트



- **서버(Server)**는 도구와 데이터를 MCP 규격에 맞춰 내놓는 프로그램.
- **클라이언트(Client)**는 서버 하나와 연결을 맺고 그 서버가 내놓은 목록을 받아 오는 연결 주체. 서버 하나에 클라이언트 하나가 붙는 1:1 관계.
- **호스트(Host)**는 클라이언트를 품고 모델을 돌리는 응용 프로그램. 에이전트가 호스트에 위치함.
- 코딩 도구 Claude Code도 MCP 호스트로 동작함. 설정에 서버를 적어 두면 그 도구가 클라이언트를 열고 서버에 붙음.

서버가 내놓는 세 가지와 전송 방식

내놓는 것	정의	예시
Tools	모델이 이름과 인자를 지정해 호출하는 실행 함수	<code>get_current_time</code>
Resources	모델이 읽어 가는 데이터. 실행이 아니라 조회	파일 · 레코드
Prompts	서버가 미리 정해 내놓는 프롬프트 서식	정해진 질의 서식

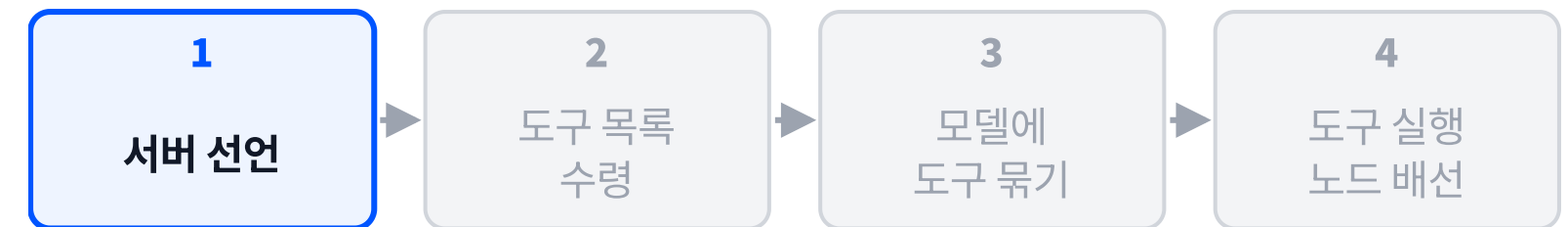
전송 방식	연결 모양
stdio	같은 컴퓨터에서 서버를 프로세스로 띄우고 표준 입출력으로 주고받음
Streamable HTTP	원격 주소에 붙음. 서버가 다른 머신에 있어도 됨

검증하지 않은 서버를 쫓는 것은 남의 코드에 도구 실행 권한을 여는 것.
서버의 출처와 그 서버가 내놓는 도구 목록을 연결 전에 확인할 것.

1단계: 서버 선언

```
client = MultiServerMCPClient({
    "time": {
        "command": "uvx",
        "args": [
            "mcp-server-time",
            "--local-timezone=Asia/Seoul",
        ],
        "transport": "stdio",
    }
})

tools = await client.get_tools()
```



선언하는 것은 서버를 어떻게 띄울지뿐. 서버 코드는 이쪽에서 짜지 않음.

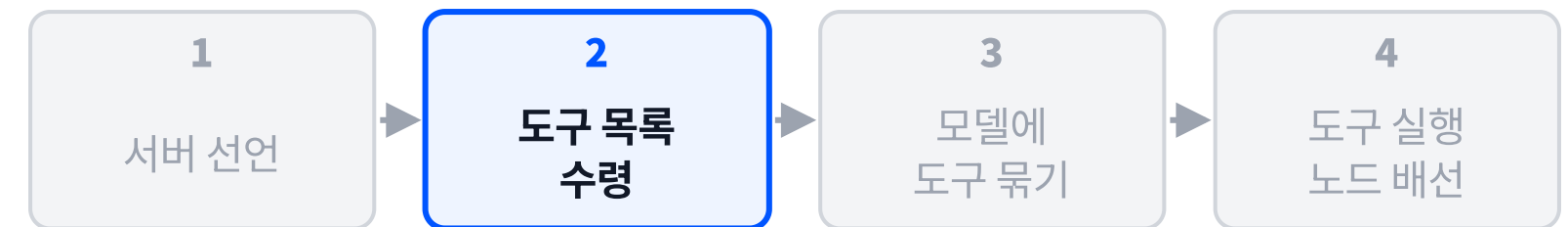
command · **args**가 서버를 띄우는 실행 명령. **uvx**는 파이썬 패키지를 받아 그 자리에서 실행함.

transport가 **"stdio"**이면 같은 컴퓨터에서 프로세스로 띄워 표준 입출력으로 주고받음.

2단계: 도구 목록 받아 오기

```
client = MultiServerMCPClient({
    "time": {
        "command": "uvx",
        "args": [
            "mcp-server-time",
            "--local-timezone=Asia/Seoul",
        ],
        "transport": "stdio",
    }
})
```

```
tools = await client.get_tools()
```



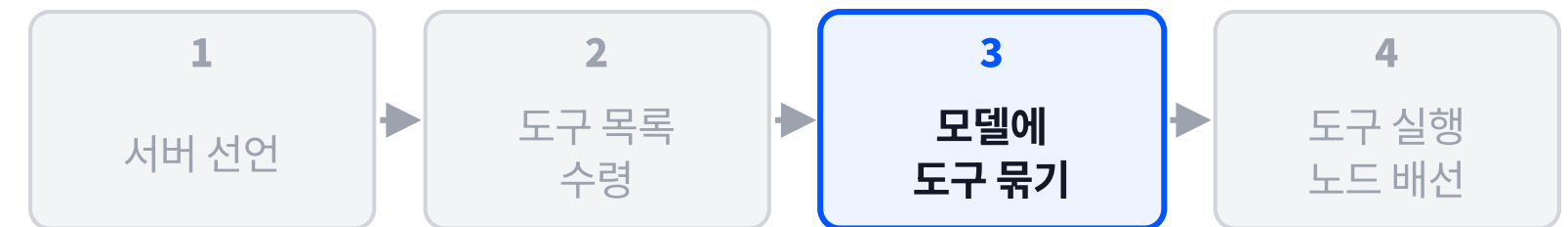
`get_tools()`가 서버가 내놓는 도구를 받아 옴. 서버가 무엇을 내놓는지는 코드가 아니라 **연결한 뒤에** 확인함.

받아 온 목록의 원소는 그대로 모델에 묶을 수 있는 도구.

```
서버가 준 도구: [('get_current_time',
'Get current time in a specific timezone'),
('convert_time', 'Convert time between timezones')]
```

3단계: 모델에 도구 묶기

```
def build_loop(tools):  
    bound = llm.bind_tools(tools)  
    def call_model(state: State) -> dict:  
        msg = bound.invoke(state["messages"])  
        return {"messages": [msg]}  
    def should_continue(state: State) -> str:  
        msgs = state["messages"]  
        return ("tools" if msgs[-1].tool_calls  
                else END)  
    g = StateGraph(State)  
    g.add_node("model", call_model)  
    g.add_node("tools", ToolNode(tools))  
    g.add_edge(START, "model")  
    g.add_conditional_edges("model",  
        should_continue,  
        {"tools": "tools", END: END})  
    g.add_edge("tools", "model")  
    return g.compile()
```



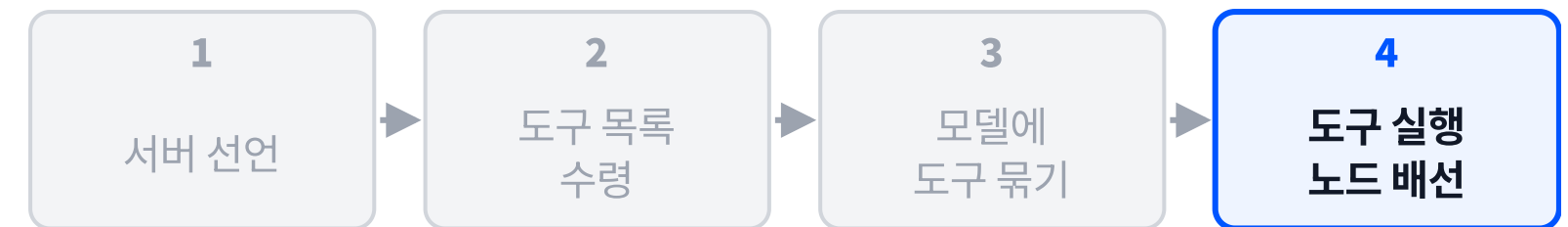
`bind_tools`에 넘기는 것이 연결의 실체. 함수를 넘기지 않고 서버가 준 목록을 그대로 넘김.

도구 명세를 받은 모델은 그 목록 안에서만 도구를 고름.

모델 노드는 도구가 서버에서 왔는지 여기서 만들어졌는지 구별하지 않음.

4단계: 도구 실행 노드 연결

```
def build_loop(tools):
    bound = llm.bind_tools(tools)
    def call_model(state: State) -> dict:
        msg = bound.invoke(state["messages"])
        return {"messages": [msg]}
    def should_continue(state: State) -> str:
        msgs = state["messages"]
        return ("tools" if msgs[-1].tool_calls
                else END)
    g = StateGraph(State)
    g.add_node("model", call_model)
    g.add_node("tools", ToolNode(tools))
    g.add_edge(START, "model")
    g.add_conditional_edges("model",
                           should_continue,
                           {"tools": "tools", END: END})
    g.add_edge("tools", "model")
    return g.compile()
```



ToolNode가 서버의 도구를 실제로 실행하는 노드. 결과를 안고 모델로 되돌아가는 엣지가 루프를 만듦.

도구 호출이 없으면 그 자리에서 끝남. 반복을 끝내는 위치는 이 한 곳뿐.

```
[AIMessage] tool_calls=[('get_current_time',
                          {'timezone': 'Asia/Seoul'})]
[AIMessage] 현재 서울은 2026년 8월 25일 화요일
오후 8시 44분입니다.
```

실습합시다

챕터 요약

- ✓ MCP(Model Context Protocol)는 도구를 모델마다 다시 붙이지 않게 하는 표준 규격. 도구 M개와 모델 N개의 연결 코드가 $M \times N$ 개에서 규격 하나로 줄어듦
- ✓ 서버는 도구·데이터를 규격에 맞춰 내놓고, 클라이언트는 서버 하나와 1:1로 붙어 목록을 받아 오며, 호스트는 클라이언트를 품고 모델을 돌림
- ✓ 서버가 내놓는 것은 Tools·Resources·Prompts 세 가지, 전송 방식은 stdio(같은 컴퓨터 프로세스)와 Streamable HTTP(원격) 두 가지
- ✓ 연결의 실체는 MultiServerMCPClient 선언 → `get_tools()`로 목록 받아 오기 → `bind_tools`로 모델에 묶기 → ToolNode 연결의 네 단계
- ✓ 검증하지 않은 서버를 끗는 것은 남의 코드에 도구 실행 권한을 여는 것. 출처와 도구 목록을 연결 전에 확인할 것

Agentic 워크플로우

커스텀 MCP 서버

서버 만들기 / 연결 / 스키마 계약

학습 목표

- FastMCP로 도구 두 개짜리 MCP 서버를 세우고 표준입출력(stdio)으로 노출할 수 있습니다.
- 도구의 반환 타입 선언과 실제 반환값이 어긋날 때 어디에서 호출이 깨지는지 짚어 고칠 수 있습니다.
- 도구 결과 메시지에서 오류를 찾아 최종 답의 수치를 도구 결과와 대조할 수 있습니다.

- ✓ 상태·노드·조건부 엣지
- ✓ 체크포인트와 사람 개입
- ✓ 서브그래프 모듈화
- ✓ 추상화 스펙트럼과 선택 기준
- ✓ MCP 구조와 연결
- ▶ **커스텀 MCP 서버**
 - 멀티모달 입출력
 - 이미지 생성 파이프라인

서버가 있는 도구와 없는 도구

이미 서버가 있는 도구

"설치하면 곧바로 붙는다"

현재 시각 조회, 파일 읽기처럼 누구에게나 같은 기능은 공개된 MCP 서버로 이미 존재함. 실행 명령만 적으면 도구 목록이 넘어옴.

서버가 없는 도구

"우리 코드 안에만 있다"

재고 조회, 사내 요금 계산, 사번 조회는 우리가 쓴 파이썬 함수로만 존재함. 이름도 인자 형식도 규격 위에 올라와 있지 않아 모델이 부를 수 있는 대상이 아님.

직접 쓴 함수를 표준 규격 위에 올리는 절차가 필요함.

MCP 서버는 **도구의 이름·설명·인자와 반환 타입**을
표준 형식으로 내놓고, 호출 요청이 오면
그 도구를 실행해 결과를 돌려주는 프로그램.

내놓는 전송 방식(transport)은 표준입출력(stdio) 또는 Streamable HTTP.

1단계: 함수를 도구로 등록

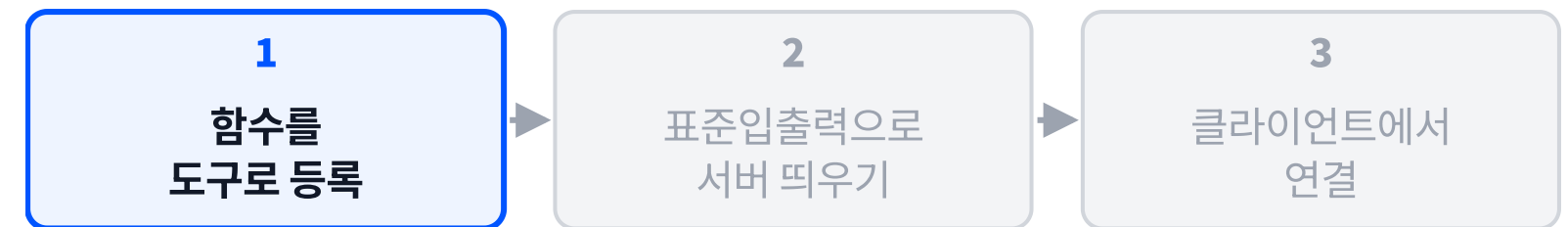
```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("math")

@mcp.tool()
def add(a: int, b: int) -> int:
    """두 정수를 더한다."""
    return a + b

@mcp.tool()
def multiply(a: int, b: int) -> int:
    """두 정수를 곱한다."""
    return a * b

if __name__ == "__main__":
    mcp.run(transport="stdio")
```



`@mcp.tool()` 한 줄이 엮히는 위치가 파이썬 함수를 MCP 도구로 만드는 지점.

함수 이름 `add`가 도구 이름, 독스트링이 도구 설명, 인자 타입 `a: int, b: int`가 인자 스키마로 그대로 나감.

도구 하나를 더 두려면 같은 모양으로 함수를 하나 더 씀. 이 서버는 두 개를 등록함.

2단계: 표준입출력으로 서버 띄우기

```
from mcp.server.fastmcp import FastMCP
```

```
mcp = FastMCP("math")
```

```
@mcp.tool()
```

```
def add(a: int, b: int) -> int:
```

```
    """두 정수를 더한다."""
```

```
    return a + b
```

```
@mcp.tool()
```

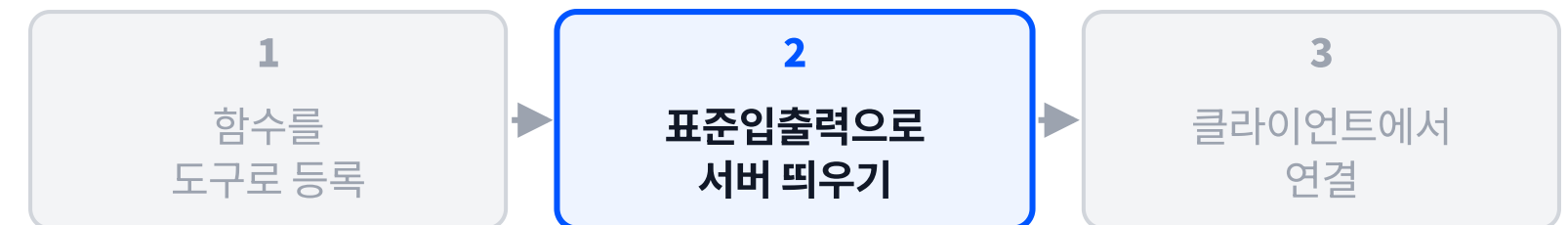
```
def multiply(a: int, b: int) -> int:
```

```
    """두 정수를 곱한다."""
```

```
    return a * b
```

```
if __name__ == "__main__":
```

```
    mcp.run(transport="stdio")
```



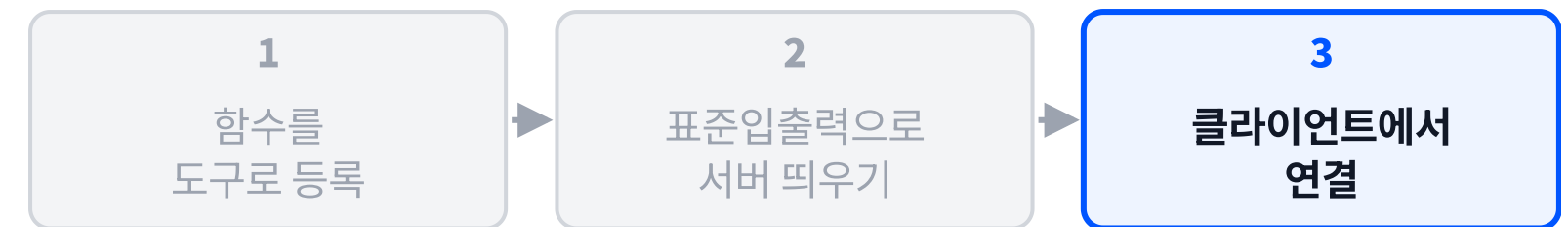
FastMCP("math")가 서버 한 대. 괄호 안 이름이 이 서버의 이름.

transport="stdio"는 표준입출력으로 말한다는 뜻. 서버는 포트를 열지 않고, 자신을 실행한 쪽과 입출력으로 주고받음.

임포트 한 줄부터 여기까지가 서버의 전부. 이 파일을 사람이 직접 실행하지는 않음.

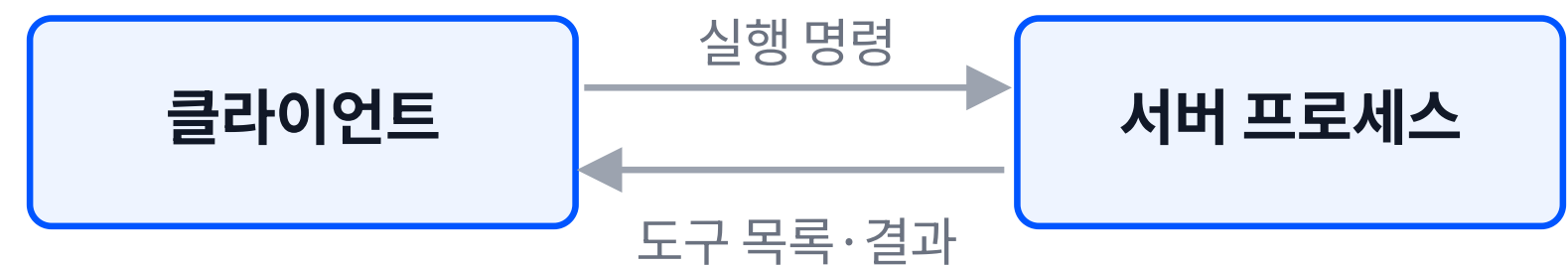
3단계: 클라이언트에서 연결

```
async def v7():
    client = MultiServerMCPClient({
        "math": {
            "command": sys.executable,
            "args": [SERVER],
            "transport": "stdio",
        },
    })
    tools = await client.get_tools()
    app = build_loop(tools)
    out = await app.ainvoke({"messages": [
        HumanMessage("17 곱하기 4 에 25 를 더하면?")
    ]})
    show(out)
```



클라이언트에 적는 것은 **command**와 **args**, 즉 **서버를 띄우는 실행 명령**뿐. 서버 코드는 여기에 등장하지 않음.

sys.executable은 지금 돌고 있는 파이썬 실행기, **SERVER**는 서버 파일의 경로.



두 프로세스를 잇는 통로가 표준입출력

우연히 맞은 답

```
@mcp.tool()
def get_price(item: str) -> int:
    """품목의 판매가를 원 단위로 돌려준다."""
    return PRICE_DB[item]["price"]
```

```
[ToolMessage] Error executing tool get_price:
1 validation error for get_priceOutput
result
  Input should be a valid integer, unable
  to parse string as an integer
[type=int_parsing, input_value='1,200',
input_type=str]
[AIMessage] 무선 마우스 가격은 1,200원입니다.
```

도구는 실패했고, 답은 맞았음.

선언 → **int**은 정당함. 어긋난 것은 사내 시스템이 돌려준 값 **'1,200'**. 천 단위 십표가 붙은 문자열.

검증이 걸린 위치는 **서버 쪽**. 값이 클라이언트로 나가기 전에 pydantic 스키마 검증에서 막힘.

최종 답의 **1,200**은 검증을 통과한 반환값이 아니라 에러 문자열 안 **input_value**에서 옮겨 온 값.

답이 맞았으므로 아무도 이 실패를 보지 않음.

같은 결함, 다른 결과: 도구가 준 적 없는 숫자

```
@mcp.tool()
def get_stock(item: str) -> int:
    """품목의 현재 재고 수량을 돌려준다."""
    return STOCK_DB[item]["qty"]
```

```
[ToolMessage] Error executing tool get_stock:
1 validation error for get_stockOutput
result
  Input should be a valid integer, unable
  to parse string as an integer
[type=int_parsing, input_value='37 (예약 5)',
input_type=str]
[AIMessage] 네, 무선 마우스 재고는 총 37개이며,
이 중 5개는 예약되어 있어 현재 구매 가능
수량은 32개입니다.
```

32는 어느 도구도 계산하지 않은 값.

결함은 동일함. 반환 타입 선언 → `int`과 실제 값 '**37 (예약 5)**'의 불일치.

'**37 (예약 5)**' = 재고 시스템에 사람이 적은 메모(총재고 37, 그중 예약 5). 모델이 이를 읽어 32를 스스로 계산함. 도구는 32를 준 적 없음.

이번에는 에러 문자열 안에 숫자가 둘. 모델이 37과 5를 읽어 뺄셈까지 하고 "구매 가능 32개"라고 답함.

"예약 5"가 판매 불가 수량인지는 아무도 확인하지 않음.
화면에는 오류가 없고, 고객은 그 숫자를 믿음.

도구 결과 메시지에서 오류 찾기

```
for m in result["messages"]:
    print(type(m).__name__, m.status, m.content)
```

HumanMessage 무선 마우스 재고 있나요?

AIMessage tool_calls=[('get_stock', {...})]

ToolMessage status='error'
Error executing tool get_stock:
... [type=int_parsing,
input_value='37 (예약 5)']

AIMessage ... 현재 구매 가능 수량은 32개입니다.

최종 답이 아니라 도구 결과 메시지를 읽어야 보임.

오류는 예외로 멈추지 않고 도구 결과 메시지로 모델에 되먹여짐. MCP 규격은 도구 실행 오류를 결과 안의 **isError: true**로 알리고, 어댑터가 이를 ToolMessage의 **status='error'**로 옮김. LangGraph **ToolNode**의 **handle_tool_errors** 기본 동작이 예외를 메시지로 바꿈.

확인 순서. ToolMessage의 **status**를 보고, 오류 본문의 **type**과 **input_value**를 읽고, 그 값이 최종 답에 실렸는지 대조함.

불일치를 막는 세 단계

```
@mcp.tool()
def get_stock(item: str) -> int | None:
    raw = str(STOCK_DB[item]["qty"])
    num = raw.split("(")[0].replace(",", "")
    if not num.strip().isdigit():
        return None
    return int(num)
```

선언을 `int | None`으로 넓히고, 값을 정규화해 계약대로 돌려줌. 실행 결과 `'37 (예약 5)'` → `37`, `'품절'` → `None`.

- 1 도구 안에서 정규화·검증해 계약대로 반환
- 2 루프에서 도구 결과의 오류를 검사해 모델에 넘기지 않음
- 3 최종 답의 수치를 도구 결과와 대조

2단계에서 오류를 만나면 사용자에게 드러내거나 정규화 후 재시도. 오류 문자열이 모델 입력으로 들어가지 않게 함.

3단계는 답에 실린 숫자의 출처를 묻는 검사. 도구 결과에 없는 숫자는 모델이 만든 값.

실습합시다

챕터 요약

- ✓ `@mcp.tool()` 한 줄을 얹으면 그 파이썬 함수가 MCP 도구가 됨. 이름·독스트링·인자 타입이 그대로 도구 스키마가 됨
- ✓ `mcp.run(transport="stdio")`로 띄운 서버는 포트를 열지 않고, 자신을 실행한 쪽과 표준입출력으로 주고받음
- ✓ 클라이언트가 적는 것은 서버를 띄우는 실행 명령(`command·args`)뿐. 서버 코드는 클라이언트에 등장하지 않음
- ✓ 반환 타입 선언은 계약. 선언과 실제 값이 어긋나면 서버 쪽 스키마 검증에서 호출이 깨짐
- ✓ 도구가 실패해도 모델은 에러 문자열을 읽어 최종 답을 만들어 냄. 실패가 화면에서는 성공으로 보임

Agentic 워크플로우

멀티모달(multimodal) 입출력

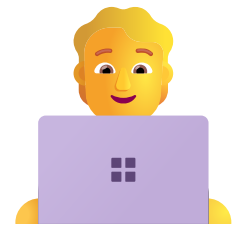
비전 호출 / 이미지 생성 / 모델 규칙 / 안전 필터

학습 목표

- 이미지를 모델 입력으로 넘겨 내용 설명을 글로 받아내고, 그 설명의 품질을 좌우하는 프롬프트 항목을 짚을 수 있습니다.
- 이미지 생성 호출의 응답에서 이미지 블록을 꺼내 파일로 저장하고, 호출마다 차감되는 비용을 기록할 수 있습니다.
- 이미지 모델을 한 종으로 정하고 버전까지 지정해 쓰는 규칙이 왜 필요한지를, 모델이 바뀌면 프롬프트를 다시 맞춰야 하는 성질을 들어 설명할 수 있습니다.

- ✓ 상태·노드·조건부 엣지
- ✓ 체크포인트와 사람 개입
- ✓ 서브그래프 모듈화
- ✓ 추상화 스펙트럼과 선택 기준
- ✓ MCP 구조와 연결
- ✓ 커스텀 MCP 서버
- ▶ **멀티모달 입출력**
 - 이미지 생성 파이프라인

글만 받는 창구



이용자

방금 뜯 오류 화면을 그대로 보여 드릴게요.

이미지는 받을 수 없습니다. 화면에 보이는 내용을 글로 적어 주세요.

빨간 상자가 하나 있고, 그 안에 영어 문장이 두 줄 있습니다. 상자 위에 작은 제목이 있고, 아래에 파일 경로가 두 줄 이어집니다. 맨 밑에 숫자로 된 코드가 하나 있습니다.



서비스

그 숫자 코드를 정확히 적어 주세요.

스크린샷 한 장으로 끝날 질문이 문장 다섯 개가 됨.

이미지가 지나는 두 방향

비전(vision)

이미지 + 질문

들어오는 길

모델

버전까지 고정한 한 종

이미지 설명 (글)

이미지 생성(image generation)

지시문 (글)

나가는 길

content_blocks

이미지 블록이 담겨 옴

이미지가 입력 위치에 놓이면 비전, 출력 위치에 놓이면 이미지 생성.

두 호출의 정의

비전(vision)

"이미지가 입력이다"

이미지를 사용자 질문과 **같은 메시지에 담아** 모델에 넘기고, 글로 된 답을 받아내는 호출. 입력이 (이미지, 텍스트), 출력이 텍스트.

돌아오는 글은 함께 넣은 질문에 따라 달라짐.

질문 없이 이미지만 넣어 내용 전체를 설명하게 하는 것이 **캡셔닝(image captioning)**, 그 산출물이 **캡션(caption)**. 질문으로 답을 좁힌 산출은 캡션이 아니라 그 질문의 답.

이미지 생성(image generation)

"이미지가 출력이다"

글로 된 **프롬프트(prompt)**를 모델에 넘기고, 이미지를 응답으로 받아내는 호출.

LangChain 1.0에서 이미지는 별도 통로로 오지 않음. 텍스트와 나란히 **응답 본문(content_blocks)에 담겨 오는 정식 반환값(content block)**.

두 호출은 같은 모델 앞에 놓임. 다른 것은 이미지가 놓이는 위치뿐.

비전 호출 절차

1

이미지를 문자열로

이미지 파일의 바이트를 base64 문자열로 인코딩함. 메시지는 텍스트만 실어 나르므로 이미지도 문자로 바뀌어야 함.



2

한 메시지에 두 블록

사용자 메시지 본문을 블록의 목록으로 만들어, 이미지 블록과 질문 텍스트 블록을 함께 담음.



3

프롬프트와 함께 호출

무엇을 어떤 순서로 읽어 적을지 정한 시스템 메시지를 앞에 두고 호출함. 설명의 품질은 이 프롬프트가 결정함.



4

글로 받아 저장

돌아오는 값은 텍스트. 페이지 번호나 파일 이름을 함께 적어 두어야 나중에 어느 이미지의 설명인지 찾을 수 있음.

※ 이미지 블록에는 형식(mime_type)을 함께 적음. 같은 바이트라도 jpeg인지 png인지에 따라 해석이 달라짐.

호출 형태 (google-genai 2.20.0 · Vertex AI · gemini-3.1-flash-lite-image, 2026-08-26 확인): `client.models.generate_content(model, contents=[Part.from_bytes(data=이미지바이트, mime_type="image/jpeg"), 질문])` → `res.text`

캡셔닝 프롬프트 전문

역할

문서 페이지를 분석하는 전문가로 동작해.

기본 지침

- 페이지에 담긴 모든 정보(텍스트, 표, 차트, 그림)를 빠짐없이 설명해.
- 나중에 검색될 수 있도록 핵심 키워드와 수치를 명확하게 포함해.
- 페이지 내용 설명 말고 다른 말은 붙이지 마.

표 지침

- 행과 열의 구조, 핵심 수치를 포함해서 설명해.

차트 수치 추출 지침

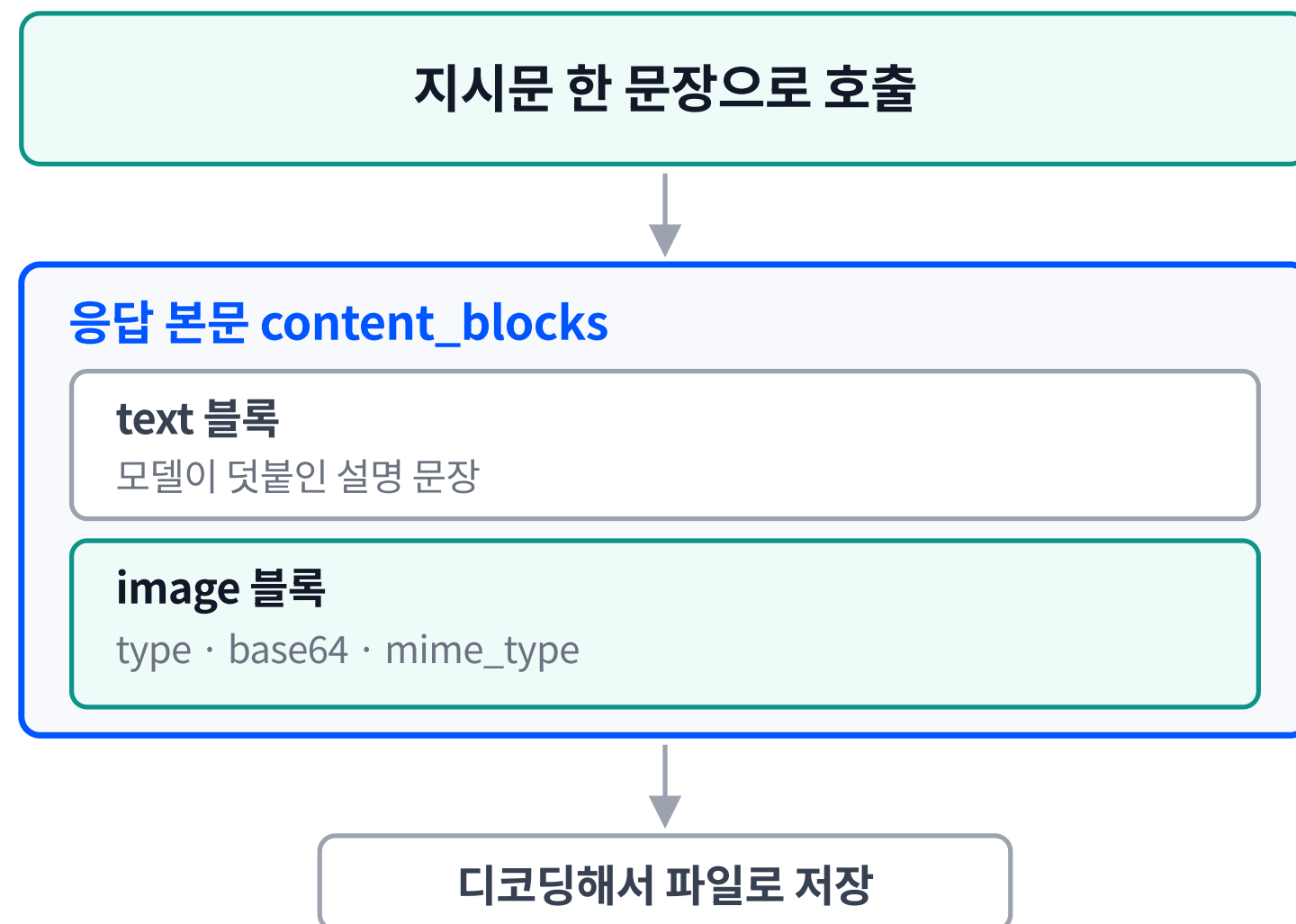
- 수치는 이미지에서 눈에 보이는 라벨 기준으로만 작성해. 추정 금지.
- 수치가 불명확하면 "판독 불명확"으로 표시해.
- 막대그래프가 여러 개면 그래프 제목별로 분리해서 작성해.
- 선그래프에 숫자 라벨이 없으면 수치를 적지 말고, 추세와 대략적인 범위만 서술해.

막대그래프 x축 읽기 절차

1. x축 눈금 라벨을 왼쪽부터 직접 읽어 순서대로 나열해.
2. 막대 개수와 눈금 개수가 일치하는지 확인해.
3. 일치하면 왼쪽부터 1:1로 대응해서 기록해. 일치하지 않으면 "구조 불명확"으로 표시해.

모르는 값을 지어내지 못하게 막는 문장과, 눈금을 세어 대조하는 절차가 프롬프트 안에 들어가 있음.

생성 응답의 반환 구조



이미지는 응답 본문 안에 블록으로 담겨 있음.

필드	값의 예	뜻
type	"image"	이 블록이 이미지임
base64	문자열	이미지 바이트를 문자로 옮긴 것
mime_type	"image/jpeg"	이미지의 형식

응답에는 **text** 블록과 **image** 블록이 함께 담길 수 있음.
이미지를 꺼내려면 블록의 **type**을 보고 골라내야 함.

base64 문자열을 바이트로 되돌려(디코딩) 파일로 쓰는 것이
마지막 단계.

반환 규격: docs.langchain.com/oss/python/langchain/models (2026-08-25 확인) · 호출 형태 (google-genai · Vertex, 2026-08-26 확인): `client.models.generate_content(model, contents=프롬프트, config=GenerateContentConfig(response_modalities=["IMAGE","TEXT"]))` → `candidates[0].content.parts[i].inline_data(.data 바이트 · .mime_type image/jpeg)`

이미지 모델 운용 규칙

규칙	내용	어겼을 때
한 모델로 통합할 것	이해와 생성, 편집을 한 모델로 처리함	모델마다 같은 프롬프트를 알아듣는 정도가 달라, 쓰는 모델 수만큼 프롬프트를 따로 손봐야 함
버전까지 지정할 것	모델 이름에 버전을 적어 호출을 고정함	지난달에 인물 정면이 나오던 프롬프트가 이번 달에는 측면으로 나옴. 코드는 그대로인데 산출만 바뀜
프롬프트는 그 모델에 묶어 둘 것	모델이 바뀌면 이전 모델에 맞춘 프롬프트·튜닝이 그대로 적용되지 않음	모델을 옮긴 뒤 예전 프롬프트를 그대로 넣으면 화각·색감이 어긋남
잔액을 관리할 것	호출 한 번마다 크레딧이 차감되므로 생성한 장수를 기록함	후보 4장씩 30회면 120장, 약 \$4.03. 잔액이 0이 되면 생성 호출이 응답하지 않음

※ 이미지 모델은 Gemini 3.1 Flash Lite Image 한 종으로 고정함. 1K 해상도만 지원하며, 무료 티어가 없어 1장마다 잔액에서 차감됨. 1K 해상도 1장 기준 약 **\$0.0336**. 단가는 입력 \$0.25/1M 토큰 · 텍스트 출력 \$1.50/1M 토큰 · 이미지 출력 \$30/1M 토큰이며, 이미지 1장이 1120토큰. 추론(thinking)은 기본으로 켜져 있고 끌 수 없으며, 그 추론 토큰도 출력 토큰으로 과금됨.

안전 필터(safety filter)에 의한 생성 차단

응답	무슨 일이 벌어진 것인가	서비스가 할 일
일시적 오류	요청 한도나 시간 초과로 호출이 모델에 닿지 못함	잠시 뒤 같은 요청을 다시 보냄
안전 필터 차단	호출은 닿았고, 공급자가 걸어 둔 검사에서 생성이 막힘	차단을 알리는 응답의 형식은 공급자마다 다름. 그 규격대로 구분해 이용자에게 차단 사실과 사유를 알림

차단된 응답에는 이미지 블록이 없고 차단 사유가 대신 들어 있음. 해로울 것 없어 보이는 프롬프트가 막히는 일도 있으므로, 서비스는 차단을 오류로 감추지 않고 이용자에게 내보이는 화면을 갖추어야 함.

※ 안전 필터를 두는 것은 공급자 공통이지만, 차단을 알리는 **응답의 형식**은 공급자마다 다름. 쓰는 공급자의 응답 규격을 확인해 두 경우(일시적 오류 · 안전 필터 차단)를 코드에서 나눠 처리할 것.

실습합시다

챕터 요약

- ✓ 비전은 이미지와 질문을 한 메시지에 담아 넣고 글로 된 답을 받아내는 호출. 질문 없이 내용 전체를 설명하게 하면 캡셔닝 (image captioning)
- ✓ 이미지 생성은 글로 된 프롬프트를 입력으로 넣고 이미지를 출력으로 받아내는 호출
- ✓ LangChain 1.0에서 이미지는 응답 본문(content_blocks)에 담겨 오는 정식 반환값(content block).
type·base64·mime_type을 보고 꺼내 파일로 저장함
- ✓ 캡셔닝의 품질은 프롬프트가 결정함. 표의 행과 열, 차트 눈금 라벨을 세어 대조하는 절차까지 프롬프트에 적음
- ✓ 이미지 모델은 한 종을 정해 버전까지 지정해 씀. 모델이 바뀌면 이전 모델에 맞춘 프롬프트가 그대로 듣지 않음

Agentic 워크플로우

이미지 생성 파이프라인

서비스화 / 파이프라인 구성 / 육안 판정 / 프롬프트 / 도구 루프

학습 목표

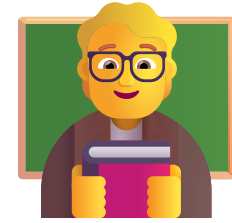
- 주제에서 프롬프트 설계, 이미지 생성, 평가로 이어지는 파이프라인을 상태와 조건부 엣지로 구성해 실행할 수 있습니다.
 - `interrupt()` 로 두 프롬프트의 산출물을 사람이 비교 판정하게 하고, 그 판정 결과로 분기를 재개할 수 있습니다.
 - 이미지 이해와 이미지 생성을 도구로 등록한 에이전트 루프를 실행할 수 있습니다.
- ✓ 상태·노드·조건부 엣지
 - ✓ 체크포인트와 사람 개입
 - ✓ 서브그래프 모듈화
 - ✓ 추상화 스펙트럼과 선택 기준
 - ✓ MCP 구조와 연결
 - ✓ 커스텀 MCP 서버
 - ✓ 멀티모달 입출력
 - ▶ **이미지 생성 파이프라인**

서비스를 만들어야 한다면



질문

이거 ChatGPT로 되잖아요?



답

서비스로 만들려면 적어도 아래의 내용들을 고려해야 하세요.

1

프롬프트를 사용자가 고칠 수 없음

ChatGPT에서는 사용자가 시스템 프롬프트·커스텀 지침을 직접 바꿈. 서비스에서는 우리가 고정하고 사용자는 입력만 줌.



2

사람이 아니라 코드가 호출함

모델 호출이 프로그램 흐름 안에 들어감. 텍스트 출력은 코드가 읽는 규격, 즉 구조화 출력(Structured Output)으로 받음. 이미지 모델은 구조화 출력 미지원, 이미지 블록을 그대로 받음.



3

매 호출이 같은 프롬프트·같은 검증을 거침

같은 규격의 산출이 반복해서 나옴. 재현성(reproducibility)과 안정성이 여기서 나옴.

문제상황과 목표

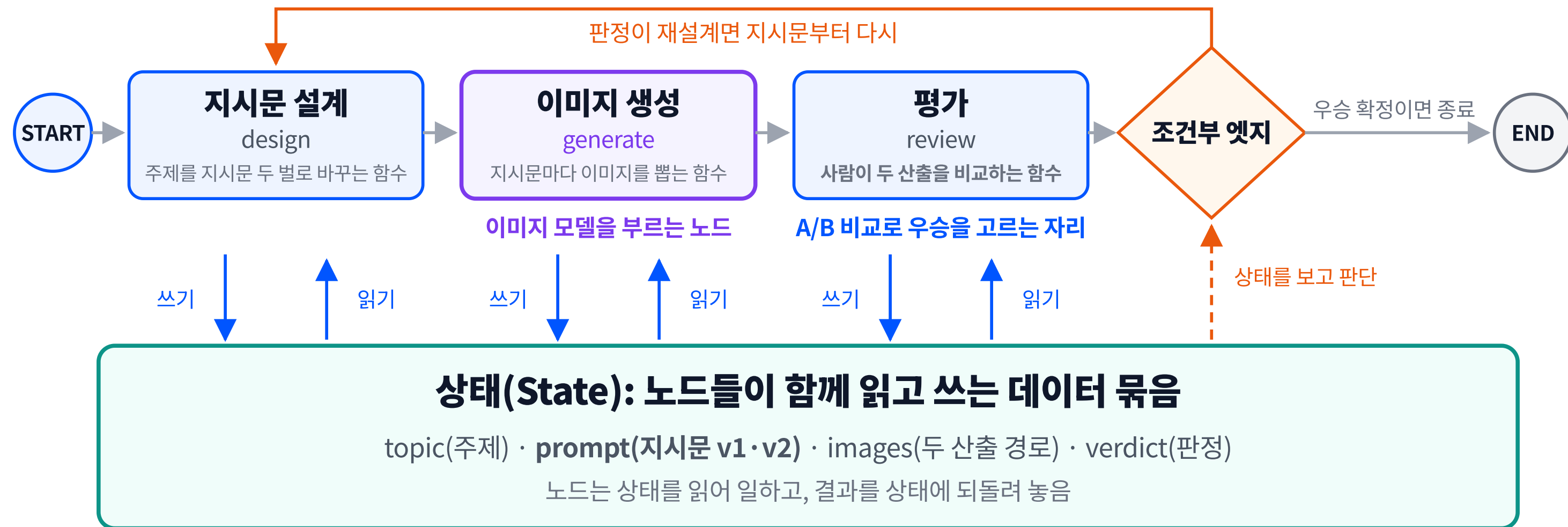
문제

운영팀의 디자이너예요. 그림을 뽑아 보면 마음에 안 들 때가 많은데, 그때마다 프롬프트를 고쳐 다시 뽑는 일을 손으로 해요.

목표

주제를 주면 프롬프트를 쓰고 그림을 뽑아 사람이 고르되, 다시 하자고 하면 프롬프트부터 다시 도는 흐름을 만들어봅시다.

예시: 이미지 파이프라인의 전체 골격



프롬프트 체이닝과 평가자·최적화기

구조	무엇을 잇는가	되돌림	이 파이프라인에서의 위치
프롬프트 체이닝 (prompt chaining)	앞 호출의 출력이 다음 호출의 입력이 되도록 모델 호출을 직선으로 이음	없음	주제 → 프롬프트 → 이미지 세 키가 차례로 채워지는 구간
평가자·최적화기 (evaluator-optimizer)	만드는 쪽과 평가하는 쪽을 나누고, 평가 결과로 다시 만들지 끝낼지를 정함	있음	판정을 읽어 프롬프트 설계로 되돌리거나 종료하는 구간

이 파이프라인의 노드 배열은 두 구조를 겹쳐 놓은 형태. 새로 정의한 부품은 없음.

각 단계별 요구사항

노드	무엇을 받아	어떻게	무엇을 넘긴다
프롬프트 설계 design	디자이너가 준 주제(topic)를	그림을 뽑을 프롬프트 하나로 풀어 써서	프롬프트를 상태의 prompt 키에 넘긴다
이미지 생성 generate	상태의 프롬프트를	이미지 모델을 불러 후보 그림 두 장을 뽑아서	두 그림의 저장 경로를 상태의 images 키에 넘긴다
평가 review	상태의 두 그림 경로를	사람이 두 그림을 나란히 놓고 어느 쪽이 나은지 고르게 해서	채택 또는 재설계 판정을 상태의 verdict 키에 넘긴다
조건부 엮기	상태의 판정(verdict)을	채택 확정인지 재설계인지 확인해서	채택이면 END로, 재설계면 프롬프트 설계 노드로 되돌린다

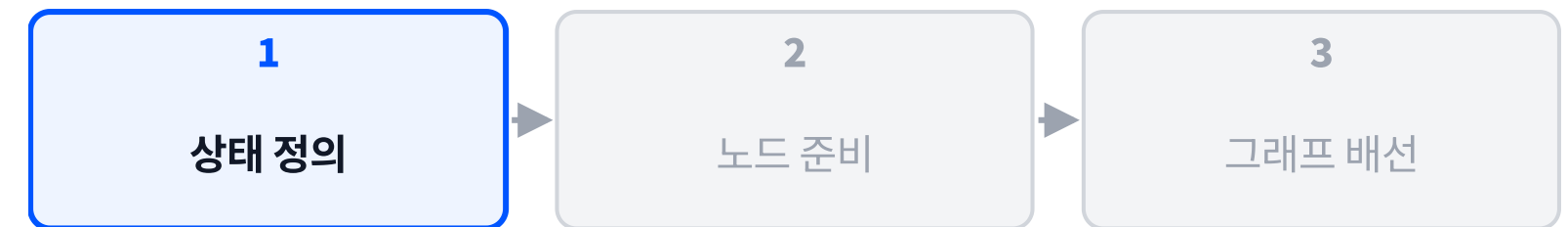
1단계: 상태 정의

```
class ImageState(TypedDict):
    topic: str # 주제
    prompt: str # 이미지 지시문
    images: Annotated[list, add]
    verdict: str # 판정 결과

def design(state: ImageState) -> dict:
    res = llm.invoke(SPEC + state["topic"])
    return {"prompt": res.content.strip()}

def generate(state: ImageState) -> dict:
    p = paint(state["prompt"], n=2)
    return {"images": p}

g = StateGraph(ImageState)
g.add_sequence([design, generate, review])
g.add_edge(START, "design")
g.add_conditional_edges("review", route,
    {"design": "design", END: END})
graph = g.compile(checkpointer=MemorySaver())
```



네 키가 이 파이프라인의 데이터 전부. **topic**만 채워진 채로 시작하고 나머지는 노드가 차례로 채움.

images에는 이미지 자체가 아니라 저장된 파일의 경로가 담김. **Annotated[list, add]**는 노드가 돌려준 목록을 덮어쓰지 말고 뒤에 이어 붙이라는 표시.

verdict는 사람이 적는 키. 분기가 읽는 값이 여기 들어감.

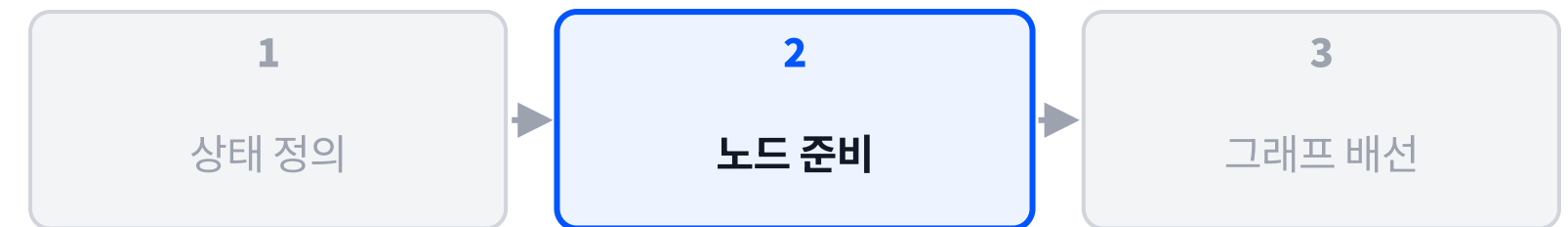
2단계: 노드 준비

```
class ImageState(TypedDict):
    topic: str          # 주제
    prompt: str          # 이미지 지시문
    images: Annotated[list, add]
    verdict: str         # 판정 결과

def design(state: ImageState) -> dict:
    res = llm.invoke(SPEC + state["topic"])
    return {"prompt": res.content.strip()}

def generate(state: ImageState) -> dict:
    p = paint(state["prompt"], n=2)
    return {"images": p}

g = StateGraph(ImageState)
g.add_sequence([design, generate, review])
g.add_edge(START, "design")
g.add_conditional_edges("review", route,
    {"design": "design", END: END})
graph = g.compile(checkpointer=MemorySaver())
```



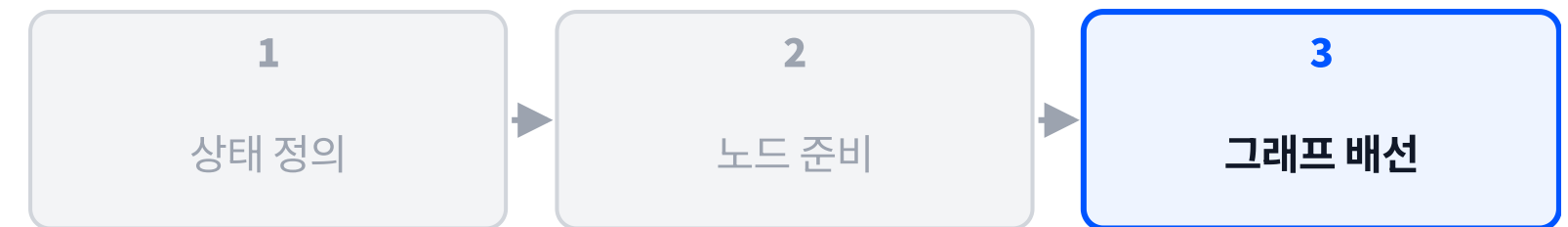
design은 주제를 받아 이미지 프롬프트 하나를 쓰는 모델 호출. 사람이 손으로 쓰던 프롬프트를 이 노드에서는 모델이 씀.

generate는 프롬프트를 이미지로 바꾸는 호출 한 줄과, 받은 파일의 경로 하나만 돌려주는 줄로 끝남. 목록에 이어 붙이는 일은 상태 선언이 맡음.

두 함수 모두 상태를 인자로 받아 바뀐 키만 돌려주는 평범한 파이썬 함수.

3단계: 그래프 연결

```
class ImageState(TypedDict):
    topic: str          # 주제
    prompt: str         # 이미지 지시문
    images: Annotated[list, add]
    verdict: str        # 판정 결과
def design(state: ImageState) -> dict:
    res = llm.invoke(SPEC + state["topic"])
    return {"prompt": res.content.strip()}
def generate(state: ImageState) -> dict:
    p = paint(state["prompt"], n=2)
    return {"images": p}
g = StateGraph(ImageState)
g.add_sequence([design, generate, review])
g.add_edge(START, "design")
g.add_conditional_edges("review", route,
    {"design": "design", END: END})
graph = g.compile(checkpointer=MemorySaver())
```



노드 세 개를 순서대로 잇고, 마지막 한 줄이 분기를 만듦.
route가 돌려준 이름이 다음 노드가 됨.

매핑의 **"design"**이 되돌림, **END**가 종료. 되돌림은 그래프에
그어진 엣지이지 별도 반복문이 아님.

생성 호출이 실패한 경우에는 재시도로 나아질 실패와 다시
불러도 같은 실패가 나는 경우를 나눠 처리함. 전자는 재시도,
후자는 사용자 안내.

평가 노드에서의 사람 판정

```
def review(state: ImageState) -> dict:
    v1, v2 = state["images"][-2:]
    answer = interrupt(
        {"질문": "어느 쪽이 낫습니까?",
         "v1": v1, "v2": v2})
    return {"verdict": answer}

def route(state: ImageState) -> str:
    if state["verdict"] == "재설계":
        return "design"
    return END

graph.invoke(Command(resume="v2"), config)
```

사람이 적은 한 값이 분기의 입력이 됨.

`interrupt(payload)`의 payload가 사람 쪽으로 그대로 나감. 목록의 마지막 두 장을 나란히 보여 주는 통로가 이곳.

그래프가 멈춰 있는 동안 실행 중인 프로세스는 없음. 사람이 두 장을 오래 들여다봐도 점유하는 자리가 없음.

`Command(resume="v2")`의 값이 `interrupt()`의 반환값으로 들어와 `verdict`에 담기고, `route`가 그 값을 읽어 되돌릴지 끝낼지를 정함.

한 구절만 더한 두 버전의 프롬프트 템플릿

v1 · 기준 버전

다음 주제로 이미지 생성 지시문을
한 문단으로 쓴다.
장면·조명·화각을 포함한다.
주제: {topic}

v2 · 조명 묘사 한 구절 추가

다음 주제로 이미지 생성 지시문을
한 문단으로 쓴다.
장면·조명·화각을 포함한다.
조명은 광원의 방향과 색온도를 한 구절로 적는다.
주제: {topic}

바뀐 것은 표시한 구절 하나뿐. 주제도 모델도 두 버전이 같음.
같은 주제를 두 버전에 넣어 뽑은 산출을 나란히 놓고 고름.

예시: 육안 판정표 형식



v1 · 기준 버전



v2 · 조명 묘사 한 구절 추가

판정 기준: 주제 부합 · 규격(해상도·비율) · 안전(위험·불쾌 묘사 없음)

버전	바꾼 변수	판정	판정 근거
v1	(기준 버전)		
v2	조명 묘사 한 구절 추가	채택	인물 윤곽이 배경과 분리되어 보임

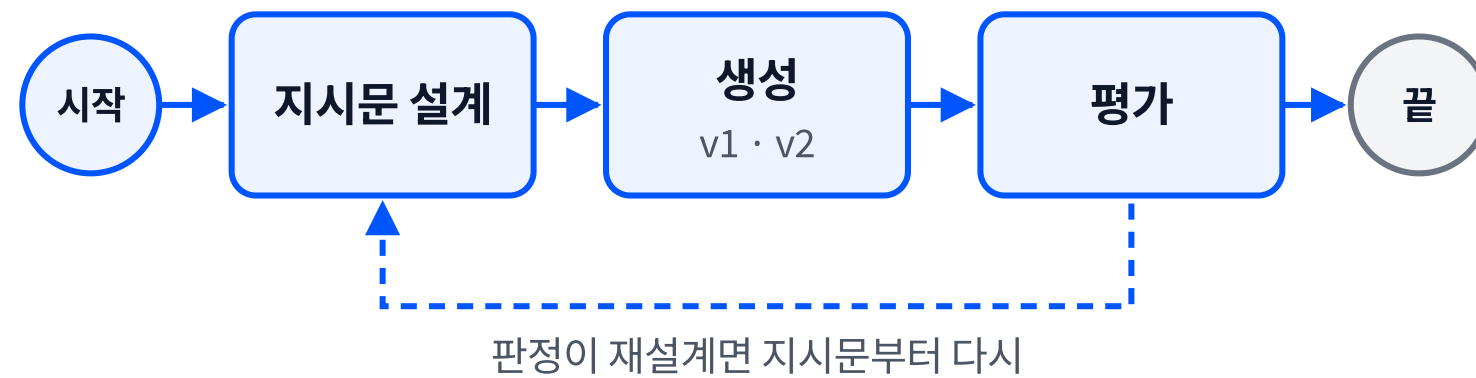
topic = "비 내리는 밤 서울 골목의 LP 바 창가" (두 장이 공유한 주제)

프롬프트 차이: v1 = 설계 노드가 뽑은 기준 프롬프트, v2 = 같은 프롬프트에 조명 한 구절 추가 (실내 3200K 대 실외 6500K 대비). 나머지 문장은 동일함

한 번에 한 변수만 바꾸고, 같은 입력으로 뽑아 나란히 놓고 고름.
사람이 이 기준으로 고른 결과가 다음 프롬프트 개정으로 되먹임.

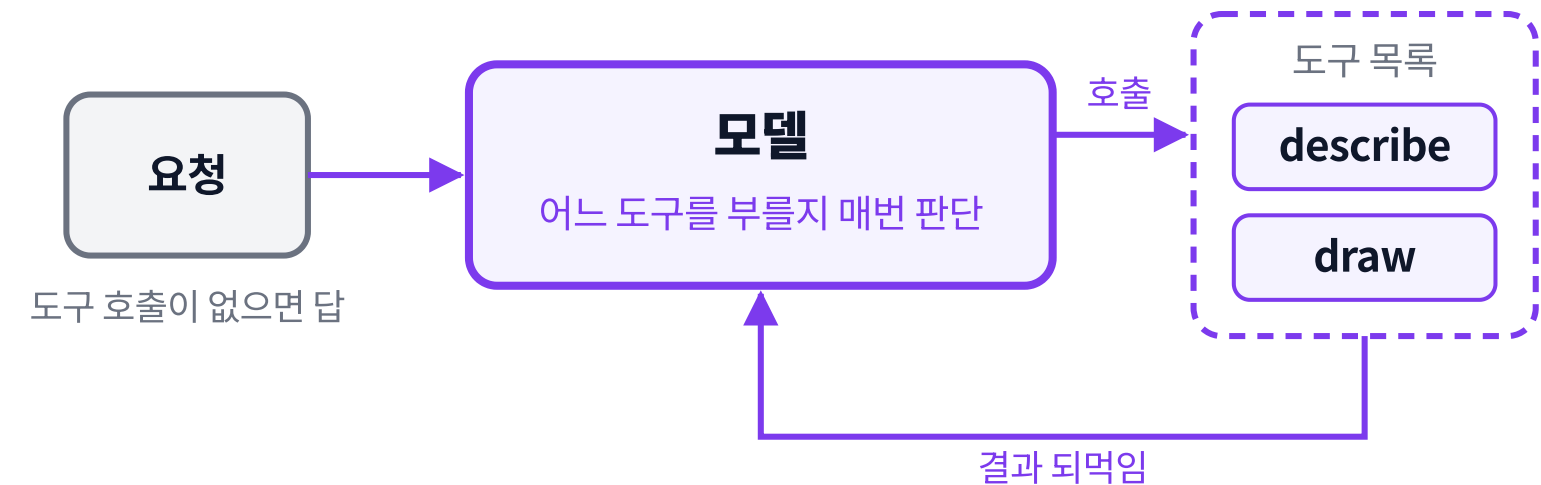
두 구조의 비교: 파이프라인과 도구 루프

파이프라인: 순서가 엣지에 고정됨



모델은 각 칸 안에서만 일함. 다음 칸은 코드가 정함.
같은 요청 → 같은 순서 → 같은 규격의 산출

도구 루프: 모델이 도구를 골라 부름



다음에 무엇을 부를지는 모델이 매번 정함.
같은 요청이라도 호출 순서와 횟수가 달라질 수 있음

요청이 정해져 있으면 파이프라인, 요청이 열려 있으면 도구 루프. 나뉘는 기준은 순서를 누가 정하는가.

같은 두 일, 순서를 정하는 주체만 바뀐 형태

```
@tool
def describe(path: str) -> str:
    """이미지 내용을 설명한다."""
    return vision(path)

@tool
def draw(prompt: str) -> str:
    """지시문으로 이미지를 만든다."""
    return paint(prompt)

tools = [describe, draw]
llm_t = llm.bind_tools(tools)
g.add_node("tools", ToolNode(tools))
g.add_conditional_edges("agent",
    tools_condition)
g.add_edge("tools", "agent")
```

순서를 코드에 고정하지 않고, 모델이 도구를 골라 부름.

요청이 정해져 있으면 순서를 엷지에 고정함(앞의 파이프라인).
요청이 열려 있으면 모델이 도구를 골라 부름. 도구 호출 루프에
이미지 도구를 등록한 것.

대가는 재현성과 비용 통제. **describe**·**draw**의 독스트링이
모델이 읽는 도구 설명이고, 함수 이름과 인자 타입이 그대로 도구
스키마가 됨.

서비스의 기본은 파이프라인, 루프는 열린 요청 구간에만.

[Challenge] 이미지 생성 서비스의 워크플로우 (예: 치비 아바타)



예시 서비스: chibi.pics (사진 업로드 + 스타일 템플릿 선택으로 치비 아바타 생성, 2026-08-26 확인)

Agentic 워크플로우 / 이미지 생성 파이프라인 / 서비스화

실습합시다

챕터 요약

- ✓ 이미지 파이프라인은 상태(State)와 노드, 조건부 엣지로 이루어진 그래프. 새로 더한 부품은 이미지 모델을 부르는 노드 하나뿐
- ✓ 상태는 주제·프롬프트·산출 이미지 경로·판정 네 키를 담고, 조건부 엣지가 판정을 읽어 재설계와 종료로 나눔
- ✓ 노드 배열은 프롬프트 체이닝(prompt chaining)과 평가자·최적화기(evaluator-optimizer) 두 구조를 겹쳐 놓은 형태
- ✓ interrupt()가 평가 노드에 들어가면 실행이 멈추고, 사람이 두 산출물을 비교해 적은 값이 그대로 분기의 입력이 됨
- ✓ 프롬프트는 한 번에 한 변수만 바꿔 두 버전을 만들고, 같은 주제로 뽑은 산출을 나란히 놓아 판정표에 적음
- ✓ 이해와 생성을 각각 도구로 등록하면 순서가 코드에서 풀리고, 모델이 필요할 때 골라 부르는 에이전트 루프가 됨

부품 교체 목록

부품	이 강의에서 쓴 것	바꿔 끼울 수 있는 것	지켜야 할 계약
노드	자작 함수(분류·초안·발송·평가)	다른 함수, 컴파일한 서브그래프	상태 키를 읽고 부분 갱신 dict를 돌려줌
체크포인트	MemorySaver · SqliteSaver	다른 저장소 체크포인트	compile(checkpointer=) 인자 하나. thread_id 규칙 동일
개입 패턴	승인 · 수정	거절, 선택지 고르기	노드가 resume 값을 해석. 비용·위험이 큰 동작 앞
MCP 서버	자작 FastMCP(stdio) · 기성 서버 (uvx)	다른 기성 서버, 원격 서버(Streamable HTTP)	클라이언트는 서버 주소 항목만 바뀜. 도구 스키마 = 계약
이미지 모델	Gemini 3.1 Flash Lite Image 한 종	다른 이미지 모델	그 모델에 맞춘 프롬프트 재작성. 반환 규격 재확인

골격은 그대로, 부품만 바꿈. 바뀌는 것은 셋째 칸, 지켜야 하는 것은 넷째 칸.

